FernUniversität in Hagen
Faculty of Mathematics and Computer Science

AIG
Artificial Intelligence Group

NC Complexity and Comparison of Algorithms for the Discussion-based Semantics in Abstract Argumentation

Bachelor's Thesis

in partial fulfilment of the requirements for
the degree of *Bachelor of Science (B.Sc.)*
in Informatik

submitted by
Vanessa Köbke

First examiner: Dr. Kai Sauerwald
Artificial Intelligence Group

Advisor: Dr. Kai Sauerwald
Artificial Intelligence Group

Statement

I declare that I have written the bachelor´s thesis independently and without unauthorized use of third parties. I have only used the indicated resources and I have clearly marked the passages taken verbatim or in the sense of these resources as such. The assurance of independent work also applies to any drawings, sketches or graphical representations. The work has not previously been submitted in the same or similar form to the same or another examination authority and has not been published. By submitting the electronic version of the final version of the bachelor´s thesis, I acknowledge that it will be checked by a plagiarism detection service to check for plagiarism and that it will be stored exclusively for examination purposes.

I explicitly agree to have this thesis published on the webpage of the artificial intelligence group and endorse its public availability.

Software created for this work has been made available as open source; a corresponding link to the sources is included in this work. The same applies to any research data.

. . . . . .

(Place, Date) (Signature)

Zusammenfassung

Diese Bachelorarbeit untersucht die theoretische Komplexität und die praktische Laufzeitperformance von Discussion-based Semantics im Kontext der abstrakten Argumentation. Das Ziel der Arbeit ist zweifach: Erstens wird gezeigt, dass die Entscheidungsprobleme EQUIVDIS und STRONGERDIS in der Komplexitätsklasse NC liegen und somit effizient parallelisierbar sind. Zweitens wird empirisch evaluiert, wie sich neu entwickelte, auf Matrix-Vektor-Multiplikationen basierende Algorithmen (MV) im Vergleich zu bestehenden Matrix-Matrix-Ansätzen (MM) schlagen und unter welchen Bedingungen Parallelisierung in der Praxis Vorteile bringt.

Auf theoretischer Ebene wird nachgewiesen, dass die Reduktionskette von EQUIVDIS über EQUALWALKCOUNT zu $\mathbb{Q}$-EQUIVALENCE in Log-Space ausgeführt werden kann. Daraus folgt die NC-Zugehörigkeit von EQUIVDIS. Ausgehend von Kiefer et al.'s Zeroness-Test für $\mathbb{Q}$-Automaten wird durch Ausnutzung der spezifischen Graphenstruktur ein deterministischer NC-Algorithmus abgeleitet. Durch den Verzicht auf unnötige Automatenkomponenten und Skalare reduziert sich das Problem auf iterierte Matrix-Vektor-Multiplikationen. Dieser Ansatz wird anschließend auf das Problem STRONGERDIS übertragen. Während der MM-Ansatz das gesamte Ranking über alle Argumente berechnet ($O(n^4)$), berechnet der MV-Algorithmus gezielt nur die für den Paarvergleich relevanten Spalten ($O(n^3)$).

In der empirischen Evaluierung auf zufallsgenerierten Daten sowie Instanzen der ICCMA 2025 zeigt sich, dass die theoretische Parallelisierbarkeit erst ab einer gewissen Eingabegröße zu praktischen Laufzeitgewinnen führt. Für die MV-Algorithmen liegt der Break-Even-Point bei ca. 100 Argumenten, da darunter der Overhead für Thread-Management dominiert. Bei größeren Instanzen erzielt die Parallelisierung Beschleunigungen um bis zu eine Größenordnung. Im Vergleich zum MM-Ansatz erweist sich der MV-Algorithmus als deutlich robuster gegenüber unterschiedlichen Graphenstrukturen und erzielt bei schwierigen Instanzen Laufzeitvorteile von mehreren Größenordnungen. Zwar stieß eine speicheroptimierte Variante auf erhebliche Laufzeitnachteile, jedoch wird deutlich, dass der Arbeitsspeicher oft das primäre Nadelöhr darstellt. Die Ergebnisse belegen, dass der MV-Ansatz eine hochgradig effiziente und zielgerichtete Alternative für Argumentvergleiche bietet.

Abstract

This bachelor's thesis investigates the theoretical computational complexity and practical runtime performance of discussion-based semantics in abstract argumentation. The study pursues two main objectives: first, establishing that the decision problems EQUIVDIS and STRONGERDIS belong to the NC complexity class, implying efficient parallelizability; and second, empirically evaluating how newly derived matrix-vector (MV) algorithms compare to existing matrix-matrix (MM) approaches and under what conditions parallel execution yields practical performance gains.

On the theoretical side, it is proven that the reduction chain from EQUIVDIS via EQUALWALKCOUNT to $\mathbb{Q}$-EQUIVALENCE can be executed in log space, establishing that EQUIVDIS is in NC. Building on Kiefer et al.'s zeroness test for $\mathbb{Q}$-automata, a deterministic NC algorithm is derived by exploiting the structural properties of graph-derived automata. Eliminating redundant automata components and random scalars yields an algorithm based directly on iterated matrix-vector multiplications, which is subsequently extended to STRONGERDIS. While the traditional MM approach computes the entire ranking across all arguments ($O(n^4)$), the MV algorithm selectively computes only the specific columns required to compare two target arguments ($O(n^3)$).

Empirical evaluations on randomly generated benchmarks and ICCMA 2025 competition instances demonstrate that theoretical parallelizability translates into practical speedups once a critical input size is reached. For MV algorithms, the parallelization break-even threshold occurs at approximately 100 arguments, below which thread management overhead dominates. On larger instances, parallel execution reduces median run times by roughly an order of magnitude. Compared to the MM approach, the MV algorithm displays significantly greater robustness to different graph structures and achieves speedup factors of several orders of magnitude on hard instances. Although a space-optimized implementation incurred severe run time penalties, the experiments highlight memory capacity as the primary bottleneck. Overall, the results confirm that MV algorithms provide a highly targeted, computationally efficient framework for pairwise argument evaluation.

Acknowledgments

I would like to thank my thesis advisor, Dr. Kai Sauerwald, for his support, availability, and encouragement, and for sending me down the rabbit hole of abstract argumentation. I would like to thank Dr. Isabelle Kuhlmann and Sandra Hoffmann for their last-minute support in categorizing the ICCMA instances, and the FernUniversität in Hagen for providing the hardware to carry out the empirical part of my work. I would also like to thank my fellow students from different study groups, without whom I would never have made it past Mathematische Grundlagen (or at least would have had a lot less fun). An especially big thank you goes to Anton, Heiko, Katrin, Kevin, Leni, Leon, Sebastian, Susanna and Tine! I thank João for the interesting discussions on algorithms. Last, but certainly not least, the biggest thank you to Patrícia for her friendship, for always believing in me, and, of course, for her keen aesthetic eye and infinite patience.

Contents

List of Figures

1 Introduction

Argumentation is concerned with the evaluation of conflicting claims and the process of determining which arguments should be considered more convincing. In computer science, abstract argumentation provides a formal framework for representing such conflicts without requiring any assumptions about the internal structure of the arguments. A central question in abstract argumentation is how the arguments of an abstract argumentation framework (AAF) should be evaluated. Classical extension-based semantics provide a discrete view of acceptability: an argument is either contained in an acceptable set of arguments or it is not. For many applications, however, this binary distinction does not capture the relative strength of arguments. Ranking-based semantics therefore assign a rank order to arguments, allowing to determine if two arguments are equally strong or if one is stronger than the other. One such approach is the discussion-based semantics proposed by Amgoud and Ben-Naim [1], which evaluates arguments based on the number of successful and unsuccessful attacks on them.

The computational complexity of ranking-based semantics is an important aspect of their practical applicability. In particular, the problems of determining whether two arguments are equally strong (EQUIVDIS) or whether one argument is at least as strong as another (STRONGERDIS) require the comparison of potentially infinite sequences of discussion counts (i.e., attacks and defenses). Recent work by Blümel et al. [5] established a threshold of $2|A| - 1$ after which algorithms to determine the ranking may halt, thereby showing that the problems are decidable in polynomial time. Their result is based on a reduction from EQUIVDIS to the problem of equivalence of $\mathbb{Q}$-automata. Since $\mathbb{Q}$-EQUIVALENCE is not only in PTime but also in NC, this reduction raises the question of whether the problems arising from discussion-based semantics can be solved in NC as well.

The NC complexity class is of particular interest from a practical perspective because it is the class of problems for which polynomial-time computations can, under suitable conditions, be efficiently parallelized. This makes an NC formulation potentially relevant for the increasingly parallel hardware available in modern computing systems. At the same time, theoretical membership in NC does not necessarily imply that a parallel implementation is faster in practice. Parallelization introduces overhead, and its benefits depend on factors such as input size, the computational structure of the algorithm, and the available hardware.

This thesis therefore investigates the discussion-based semantics from both a theoretical and an empirical perspective. First, it extends the existing complexity result by showing that the reduction from EQUIVDIS to $\mathbb{Q}$-EQUIVALENCE can be carried out in log space. This establishes that EQUIVDIS is in NC. Based on Kiefer et al.'s algorithm [17] for testing zeroness of $\mathbb{Q}$-automata, an NC algorithm for EQUIVDIS is then derived and subsequently simplified and optimized by exploiting the particular structure of the automata resulting from the reduction. Building on this algorithm, an NC algorithm for STRONGERDIS is developed as well.

Second, the thesis investigates the practical run-time implications of these theoretical results. The developed NC algorithms and the PTime algorithms proposed by Blümel et al. are implemented in Java in sequential and parallel variants. Their performance is evaluated experimentally with respect to input size, parallelization and memory requirements. In particular, the experiments investigate when parallelization becomes beneficial and whether the theoretically lower time complexity of the NC algorithms translates into an observable practical advantage. The empirical evaluation is complemented by experiments on instances from the ICCMA 2025 competition in order to consider different kinds of graph structures.

The remainder of this thesis is structured as follows. Section 2 introduces the necessary background on abstract argumentation, ranking-based semantics, and the computational complexity of discussion-based semantics, and establishes the goals of the thesis. Section 3 proves that EQUIVDIS is in NC and derives optimized NC algorithms for EQUIVDIS and STRONGERDIS. Section 4 describes the implementation of the different algorithms and the experimental methodology. Section 5 presents and discusses the empirical results. Section 6 concludes.

2 Background and Goals

In this section, the basic concepts and ideas necessary to the work in this thesis will be introduced. These are fundamentally Abstract Argumentation Frameworks as proposed by Dung [11], and ranking-based semantics for AAF. Furthermore the current state of research concerning the complexity of ranking-based semantics for AAF will be discussed. The section concludes with establishing the goals which this work aims to accomplish.

2.1 Abstract Argumentation Frameworks

Argumentation, i.e., the process of weighing different arguments against each other with the goal of establishing some kind of "truth" or at least a conclusion on which argument is more convincing [30], has been studied by mankind since the time of ancient Greek philosophers [11, 19, 30]. Argumentation is dialectical, in the sense that it can be viewed as a dialogue between a proponent who presents one or more arguments and an opponent who seeks to refute those arguments [30]. Furthermore, it is a form of non-monotonic reasoning as opposed to monotonic forms of reasoning, such as mathematical proofs, which always hold once they have been established as correct. An argument, on the other hand, is in nature defeasible [4, 19]. In modern days, argumentation is still a relevant field of study in disciplines such as legal reasoning, computer science, and artificial intelligence [4, 19, 30].

Walton [30] defines an argument as "a set of statements (propositions), made up of three parts, a conclusion, a set of premises, and an inference from the premises to the conclusion". Arguments can be independent of one another, such as in a discussion about the weather as in Example 1, where the two arguments can be either true or false, independently of the truth value of the other argument. However, in cases such as Example 2 the two arguments contradict each other, and both cannot be true at the same time. This situation will be called an attack of argument 1 on argument 2, and viceversa [30].

Example 1. Let there be the following arguments about the weather.

1. It's raining.
2. It's cold.

Example 2. Let there be the following arguments on whether it is raining.

1. The sky is blue, thus it cannot be raining.
2. Water is falling from above, thus it must be raining.

One might argue that the internal structure of argument 2 of Example 2 is not very plausible to begin with. However, in Abstract Argumentation (as well as this thesis) the internal structure of arguments will not be considered, and arguments will be

Figure 1: The argumentation graphs for Examples 1 (left) and 2 (right).

viewed as atomic [4,11]. Instead of studying the internal structure of arguments, abstract argumentation frameworks after Dung [11] and subsequent extension-based semantics consider the relationship (i.e., attacks) between arguments, and answer the question of which sets of arguments are jointly admissible. Dung defines an abstract argumentation framework as follows:

Definition 1 (Abstract argumentation framework)**.** An abstract argumentation framework is a tuple $F = (A, R)$ with A being a set of arguments, and $R \subseteq A \times A$ being the attack relation between arguments.

AAFs can be interpreted as directed graphs with A being the vertices and R being the edges. Examples 1 and 2 can be represented as shown in Figure 1.

The original question abstract argumentation aims to answer is which argument or set of arguments is the plausible outcome of an argumentation. For this purpose Dung [11] defines the concepts of admissibility and acceptability[1].

Definition 2 (Acceptability and admissibility [11])**.** Let $F = (A, R)$ be an AAF, with $a, b, c \in A$ being arguments in F and $S \subseteq A$ a subset of arguments.

1. The argument a is acceptable with regard to S, iff for each argument b that attacks a, there is an argument $c \in S$ that attacks b. This is also called c defends a.

2. A set of arguments S is considered conflict-free if there are no two arguments $a, b \in S$ such that a attacks b.

3. S is an admissible set of arguments, iff S is conflict-free and each argument in S is acceptable with regard to S.

It can be seen that the set {1, 2} in Example 1 is conflict-free, whereas in Example 2 it is not. In Example 1, all subsets of A, i.e., $\{\{1, 2\}, \{1\}, \{2\}, \emptyset\}$ are admissible, since the attack relation is empty, and therefore all arguments are acceptable. In Example 2, however, the situation is more complex. Argument 1 is attacked by argument 2, while simultaneously attacking argument 2 and thus defending itself. Let us consider the following modification to Example 2.

[1] In this thesis the distinction between different extension-based semantics, such as grounded, stable or preferred [4, 11], will not be considered.

Example 3. Let there be the following arguments.

1. The sky is blue, thus it cannot be raining.
2. Water is falling from above, thus it must be raining.
3. The neighbor's sprinkler is on.

The corresponding AAF can be expressed as follows:

$$F = (\{1, 2, 3\}, \{(1, 2), (2, 1), (3, 2)\})$$

We can see that now $S_1 = \{1, 3\}$ is an admissible subset, since the only argument that attacks elements of S (the argument 2) is attacked by 1 and 3. However $S_2 = \{2\}$ is no longer an admissible subset, because 2 is attacked by 3 without there being any argument in S_2 (or F for that matter) to defend 2.

2.2 Ranking-based Semantics

In the previous section, abstraction argumentation frameworks have been introduced. The different extension-based semantics provide a binary solution to the question of which subsets of arguments from a given set of arguments are admissible [4]. For certain practical purposes, such as decision-making, this binary view may be too simplistic to address more nuanced problems where information about the relative strength of arguments is more useful than a discrete answer about their acceptability [1, 6]. More fine-grained approaches were therefore developed (see Bonzon et al.'s work for an overview [6]).

Instead of providing a binary solution, ranking-based semantics return a ranking between arguments indicating the relative strength of the arguments [1]. Amgoud and Ben-Naim [1] point out the following differences between previous extension-based approaches and ranking-based semantics:

- In extension-based semantics, an argument is killed if it is attacked and the attacker cannot itself be defeated. In ranking-based semantics, an argument is weakened by attacks but not necessarily killed.
- Ranking-based semantics allow to differentiate between an argument with one or 55 successful attackers. In extension-based semantics, in both cases the argument would simply be not acceptable.
- In extension-based semantics, the evaluation of an argument is absolute and discrete (acceptable or not), whereas, in ranking-based semantics, it is relative and continuous (different degrees of acceptability).
- Extension-based semantics provide information about which subsets of arguments within an AAF are acceptable collectively, whereas, in ranking-based semantics, the relative strength of arguments is considered individually[2].

[2]This last difference is pointed out in [6].

Definition 3 (Ranking [1])**.** A ranking $\succeq$ on a set A of arguments is a binary, total and transitive relation on the arguments in A, where $a \succeq b$ means that a is at least as acceptable as b.

For easier notation $a \simeq b$ will be used if $a \succeq b$ and $b \succeq a$ hold[3]. Amgoud and Ben-Naim [1] furthermore present a series of postulates by which different ranking-based semantics can be classified (see also [6] for additional properties and a comparison of different ranking-based semantics). A more detailed consideration of these postulates lies beyond the scope of this thesis, therefore, they will only be briefly and informally mentioned in order to help characterize discussion-based semantics.

1. Abstraction: The internal structure of arguments is not considered. Therefore, iff two AAF F and F' have isomorphic graphs, the resulting rankings of F and F' will be the same as well.

2. Independence: The ranking among a subset of arguments is only influenced by arguments which are connected to those arguments. Unconnected arguments do not influence that ranking.

3. Void precedence: An unattacked argument is stronger than an attacked argument.

4. Defense precedence: An attacked but defended argument is stronger than an attacked but undefended argument.

5. Counter-transitivity: If the attackers of an argument b are at least as strong as the attackers of an argument a, then a is at least as strong as b.

6. Strict counter-transitivity: If the attackers of an argument b are strictly stronger than the attackers of an argument a, then a is strictly stronger than b.

7. Cardinality precedence: The strength of an argument is evaluated based on the number of its attackers.

8. Quality precedence: The strength of an argument is evaluated based on the quality of its attackers.

9. Distributed-defense precedence: If two arguments have the same number of attackers and defenders respectively, the argument whose defenders are more evenly spread across the attackers is stronger.

Most of these postulates are compatible among each other, and some even entail one another. However, crucially, cardinality precedence and quality precedence cannot be satisfied simultaneously in a given semantics. Simply put, one has to choose

[3]Note that in contrast to Amgoud and Ben-Naim's notation in this thesis the $a \succeq b$ notation from [5,6] indicating that a is at least as acceptable as b, will be used.

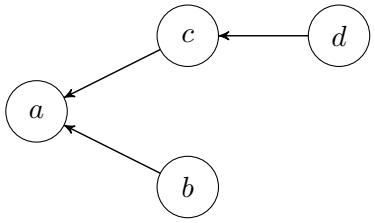

Figure 2: AAF $F = (\{a, b, c, d\}, \{(b, a), (c, a), (d, c)\})$.

whether the number of attackers or the quality of attackers is considered more relevant[4]. Both options can be rational [1].

In the remainder of this thesis, the discussion-based semantics proposed by Amgoud and Ben-Naim [1], as one type of ranking-based semantics, will be considered[5]. Discussion-based semantics is an instance of a semantics that satisfies the cardinality precedence postulate. The relevant measure for the strength of argument is the discussion count, i.e., the number of linear discussions of odd or even length.

Definition 4 (Linear discussion [1])**.** Let $F = (A, R)$ be an AAF. A linear discussion in F is a sequence of arguments $s = \langle x_1, x_2, ..., x_n \rangle$ with $x_i \in A$ for all $1 \leq i \leq n$ and $(x_{i+1}, x_1) \in R$. A linear discussion is won if n is odd; it is lost if n is even.

In graph-theoretic terms, a linear discussion corresponds to a walk ending in the vertex x_1. The relevance of the parity of the length of the linear discussion (or walk) immediately becomes clear if we consider argument $x_1 = a$ in Figure 2. The linear discussion $\langle a, b \rangle$ with $n = 2$ is lost, because argument a is attacked by b, and b in turn is unattacked. The linear discussion $\langle a, c, d \rangle$ with $n = 3$, on the other hand, is won, because even though a is attacked by c, c itself is attacked by the unattacked argument d, making d a defender of a.

Definition 5 (Discussion count [1])**.** For an AAF $F = (A, R)$ and an argument $a \in A$, the discussion count

$$Dis_i(a) = \begin{cases} -N & \text{if i is odd} \\ N & \text{otherwise} \end{cases}$$

with N being the number of linear discussions of length i for a in f.

Amgoud and Ben-Naim [1] define their discussion-based semantics as follows:

[4]Note that nevertheless there are semantics which take both quality and quantity of attackers into consideration [6].

[5]This particular instance of ranking-based semantics is chosen because this thesis aims to provide a run-time analysis for the algorithm proposed in [5], and their algorithm is based on Amgoud and Ben-Naim's discussion-based semantics. Furthermore, discussion-based semantics seems to be a good example of ranking-based semantics, since empirically it provides rankings very similar to many other kinds of ranking-based semantics [6].

Definition 6 (Discussion-based semantics [1])**.** For an AAF $F = (A, R)$ and argument $a, b \in A$, $a \succeq b$ holds if one of the two following cases apply:

1. For all $i \in \mathbb{N},\ Dis_i(a) = Dis_i(b)$ holds.
2. There exists an $i \in \mathbb{N}$ such that $Dis_i(a) < Dis_i(b)$ holds, and for all $j \in \{1, 2, ..., i-1\} Dis_j(a) = Dis_j(b)$ holds.

Example 4. If we consider the AAF in Figure 2, the discussion counts illustrated in Table 1 can be calculated. Only i up to 2 need to be considered, because there is no walk longer than 2.

We can observe the following:

- $b \simeq d$ holds, because case (1) from Definition 6 applies.
- $b, d \succeq a, c$ holds, because case (2) from Definition 6 applies for $i = 1$.
- $c \succeq a$ holds, because case (2) from Definition 6 applies for $i = 1$.

The rank order for F is therefore $b \simeq d \succeq c \succeq a$, indicating that a is the weakest argument, and c and d are the strongest with no difference between the two.

Beyond establishing the rank order on an AAF, two related problems can be derived. These problems are common to different types of ranking-based semantics. In continuation, however, EQUIVDIS and STRONGERDIS are meant to denote the problems applied to the discussion-based semantics introduced by Amgoud and Ben-Naim [1].

Problem 1. STRONGERDIS
Input: An AAF $F = (A, R)$ and two arguments $a, b \in A$.
Output: Yes, if $a \succeq b$ holds; otherwise no.

Problem 2. EQUIVDIS
Input: An AAF $F = (A, R)$ and two arguments $a, b \in A$.
Output: Yes, if $a \simeq b$ holds; otherwise no.

Since an AAF may contain cycles, the number of walks to be considered may potentially be infinite. Amgoud and Ben-Naim [1] conjecture the existence of a threshold after which the number of walks only evolves cyclically, and therefore would not need to be considered to establish the ranking between the arguments. Blümel

i	a	b	c	d
1	2	0	1	0
2	1	0	0	0

Table 1: Dis_i for the AAF F from Figure 2.

et al. [5] show that this is indeed the case by reducing EQUIVDIS to the equivalence problem of $\mathbb{Q}$-automata. They prove that the threshold t is $2n - 1$ with n being the number of arguments in the AAF. The importance of their finding for this thesis needs to be stressed; without a threshold, potential algorithms for solving EQUIVDIS and STRONGERDIS might not terminate, and the problems might not be decidable for cyclic AAFs.

2.3 Complexity of discussion-based Semantics

As mentioned in the previous section, Blümel et al. [5] show that for the discussion-based semantics proposed by Amgoud and Ben-Naim [1], a threshold exists after which the result does not change, hereby confirming that algorithms to solve STRONGERDIS and EQUIVDIS do exist. Previous empirical work on discussion-based semantics had been conducted using different thresholds. Delobelle [8] and Bonzon et al. [6] use the threshold of $|A|^2 + 1$ with A being the set of arguments, stating that they "conducted an empirical study that led [them] to conclude that this limit seemed to work well. However, [they] were convinced that there might be a better choice to be found" [9]. Jawhari [14] uses a similar algorithm to the one proposed by Blümel et al. [5], however, considers only $|A|$ iterations, while acknowledging that the ranking might still evolve after this number of iterations.

Blümel et al. [5] show that a valid threshold is $2|A|-1$, proving that these problems are decidable in PTime. Their proof uses a two-step polynomial reduction chain, whose components EQUALWALKCOUNT and $\mathbb{Q}$-EQUIVALENCE are defined below.

Problem 3. EQUALWALKCOUNT [5]
Input: A graph $G = (V, E)$ and two vertices $v_1, v_2 \in V$.
Output: Yes if v_1 and v_2 agree in the number of walks; no otherwise.

In order to define $\mathbb{Q}$-EQUIVALENCE, we must first define $\mathbb{Q}$-automata.

Definition 7 ($\mathbb{Q}$-automaton ([5])**.** A $\mathbb{Q}$-automaton is defined as $\mathcal{A}_G = \{Q, \Sigma, \delta, I, F\}$ with

- Q: a finite set of states
- Σ: the alphabet
- $\delta : Q \times \Sigma \times Q \to \mathbb{Q}$: the transition function
- $I : Q \to \mathbb{Q}$: the initial states
- $F : Q \to \mathbb{Q}$: the accepting states

$\mathbb{Q}$-EQUIVALENCE is the equivalence problem for two $\mathbb{Q}$-automata as defined below.

Problem 4. $\mathbb{Q}$-EQUIVALENCE ([5])
Input: Two $\mathbb{Q}$-automata $\mathcal{A}_1$, $\mathcal{A}_2$ over the same alphabet.
Output: Yes if $\mathcal{A}_1$ and $\mathcal{A}_2$ are equivalent, no otherwise.

In order to prove the decidability of discussion-based semantics and the validity of their threshold, Blümel et al. [5] establish the following polynomial reduction chain.

$$\text{EQUIVDIS} \leq_p \text{EQUALWALKCOUNT} \leq_p \mathbb{Q}\text{-EQUIVALENCE}$$

The main components and broad idea of their proof are outlined below. By providing a polynomial reduction of EQUIVDIS to a problem in PTime, it can be proven that EQUIVDIS is in PTime as well.

Proposition 8. $\mathbb{Q}$-EQUIVALENCE *is in NC [16, 29].*

The problem selected is $\mathbb{Q}$-EQUIVALENCE, which is in the complexity class NC. As the problems in NC are a subset of the PTime problems, $\mathbb{Q}$-EQUIVALENCE is in PTime.

Proposition 9 (Complexity classes)**.** *The following relationship between complexity classes holds [3, 18, 27] :*

$$L \subseteq NC \subseteq RNC \subseteq PTIME$$

Combining these elements, they prove that EQUIVDIS is in PTime.

Proposition 10. EQUIVDIS *is in PTime.*

Proof. [5]

- The following reduction chain holds:
  $\text{EQUIVDIS} \leq_p \text{EQUALWALKCOUNT} \leq_p \mathbb{Q}\text{-EQUIVALENCE}$ [5].
- Due to the polynomial reducibility and Propositions 8 and 9, EQUIVDIS is in PTIME.

□

Before concluding this section, the three problems and the results of the reduction will be illustrated using the initial Example 3 about the weather (copied below for convenience).

Example 5. Let there be the following arguments.

1. The sky is blue, thus it cannot be raining.
2. Water is falling from above, thus it must be raining.
3. The neighbor's sprinkler is on.

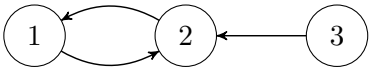

Figure 3: The argumentation graph for Example 5.

EQUIVDIS: Are arguments 1 and 2 equally strong?
As was established in previous sections, argument 1 is stronger than argument 2 because argument 1 is only attacked by one argument, whereas 2 is attacked by two arguments. Thus, 1 is stronger than 2.
The corresponding AAF $F = (\{1, 2, 3\}, \{(1, 2), (2, 1), (3, 2)\})$ can be represented as the graph in Figure 3.
EQUALWALKCOUNT: Do vertices 1 and 2 agree in the number of walks?
Table 2 illustrates the walks of a given length ending in vertices 1 and 2. It is obvious that the two vertices do not agree in the number of walks.

The problem can be converted into the following two $\mathbb{Q}$-automata. Figure 4 illustrates the resulting automata $\mathcal{A}_1$ and $\mathcal{A}_2$, with the accepting states in double circles. All states are starting states and the transitions with zero weight are not drawn.

- $\mathcal{A}_1$ with
  - states: 1, 2, 3
  - alphabet: s
  - transition matrix: $\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$
  - initial states: 1, 2, 3
  - final states: 1
- $\mathcal{A}_2$ with
  - states: 1, 2, 3
  - alphabet: s
  - transition matrix: $\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$
  - initial states: 1, 2, 3

Length	Argument 1	Argument 2
1	1	2
2	2	1
3	...	...

Table 2: Walks in Graph 3.

– final states: 2

$\mathbb{Q}$-EQUIVALENCE: Are the automata $\mathcal{A}_1$ and $\mathcal{A}_2$ equivalent?

While the answer to the corresponding EQUIVDIS and EQUALWALKCOUNT problems could be derived by knowledge provided thus far, $\mathbb{Q}$-EQUIVALENCE is more complex and an algorithm to solve this problem will be discussed in Section 3.2. For now let it suffice to know that the two automata are not equivalent.

2.4 Goals

As discussed in the previous sections, Blümel et al. [5] show that STRONGERDIS and EQUIVDIS are decidable in PTime by providing a polynomial reduction of EQUIVDIS to $\mathbb{Q}$-EQUIVALENCE. As $\mathbb{Q}$-EQUIVALENCE is in PTime, this reduction proves that EQUIVDIS is in PTime. However, $\mathbb{Q}$-EQUIVALENCE is not only in PTime but also in NC [16]. Blümel et al.'s work therefore hints at the possibility that EQUIVDIS might also be in NC. The first goal of this bachelor's thesis is to prove that this is indeed the case, by providing a log space version of the given reduction chain. Furthermore, this thesis aims to develop NC algorithms to solve STRONGERDIS and EQUIVDIS, derived from Kiefer et al.'s [17] algorithm to solve zeroness for $\mathbb{Q}$-automata.

While there has been extensive research on ranking-based semantics, most work focuses on the theoretical properties of ranking-based semantics and their implications on the resulting ranking. Some works consider the computational complexity [10] from a theoretical point of view, however, few studies have carried out experiments. Bonzon et al. [6], for example, undertake an empirical and comparative study on different types of ranking-based semantics, computing the rankings of 10,000 randomly generated AAFs. However, their paper does not provide a runtime analysis of their computation. This fact is understandable, as until the publication of Blümel et al.'s work no proven threshold after which computation can terminate in case of cyclic AAFs existed, and all experiments would therefore have to use a somewhat arbitrary threshold. In consequence, little is known of the actual empirical run time of discussion-based semantics.

Therefore, the second goal of this thesis is to provide insight in the empirical computational complexity of discussion-based semantics, as one representative of

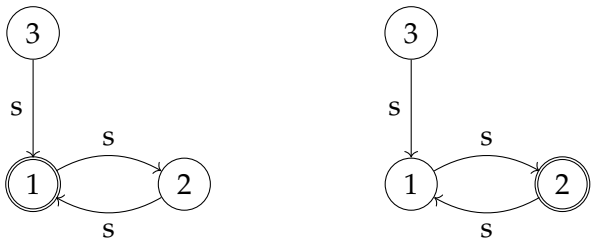

Figure 4: The $\mathbb{Q}$-automata $\mathcal{A}_1$ (left) and $\mathcal{A}_2$ (right).

ranking-based semantics. NC is the class of problems that can be efficiently parallelized [3]. Therefore, the effect of parallelization on the developed NC algorithms and Blümel et al.'s algorithms will be studied comparatively by means of an empirical experiment. For this purpose, the aforementioned algorithms will be implemented in different versions in the programming language Java. The experiment will contain an optimization layer, using randomly generated input, and culminate in the comparison of the optimal implementations, using the ICCMA 2025 [2] test instances as input.

3 NC complexity of EQUIVDIS

The goal of this section is to prove that EQUIVDIS is in NC, by providing a log space reduction from EQUIVDIS to a problem in NC, namely $\mathbb{Q}$-EQUIVALENCE. Furthermore, an algorithm to solve this problem in NC will be outlined.

3.1 Reduction of EQUIVDIS to Q-EQUIVALENCE

As discussed in the previous section, Blümel et al. [5] propose the following reduction chain: EQUIVDIS $\leq_p$ EQUALWALKCOUNT $\leq_p$ $\mathbb{Q}$-EQUIVALENCE. In order to prove that EQUIVDIS is in NC, the same reduction steps will be used. However, it will be shown that each reduction step can be computed not only in polynomial time, but also in log space. For this purpose, the concept of a log space transducer as defined by Sipser [27] will be used.

Definition 11 (Log space transducer [27])**.** A log space transducer is characterized by having

- a read-only input tape,
- a write-only output tape whose head can only move rightward
- a read/write work tape, which is bound to a length of $O(\log n)$ symbols.

A log space transducer computes a log space computable function $f : \Sigma^* \to \Sigma^*$ with w being the initial word on the input tape, and $f(w)$ the word on the output tape after the log space transducer halts.

Crucially, only the work tape is bound to $O(\log n)$, and only the output tape head is limited to rightward movement. Both the input and the output tapes' length is unbound, and both the input and work tapes' heads can move in both directions as required.

Definition 12 (Log space reducibility [27])**.** Language A is log space reducible to language B, written A $\leq_l$ B, if A is mapping reducible to B by means of a log space computable function f.

With the concept of log space reducibility introduced in Definition 12, the reduction chain can now be established.

Theorem 13. EQUIVDIS $\leq_l$ EQUALWALKCOUNT

Proof. Blümel et al. [5] prove that the identity function

$$(F, a, b) \mapsto (F, a, b)$$

is a valid polynomial reduction. The identity function can be easily computed by a log space transducer, by reading each symbol from the input tape and writing it on the output tape. The $O(\log n)$ work tape is not required, therefore the reduction can be carried out in log space. □

Before proceeding to the second step of the reduction chain, the concept of a specific $\mathbb{Q}$-automaton derived from a graph G and a vertex of this graph, as used by Blümel et al. [5] is introduced.

Definition 14 (The $\mathbb{Q}$-automaton $\mathcal{A}_G^v$ [5])**.** For a given graph $G = (V, E)$ and a vertex $v \in V$, the derived $\mathbb{Q}$-automaton is defined as $\mathcal{A}_G^v = \{V, \Sigma, \delta, I, F\}$ with

$$\begin{aligned} V &= \text{ the states} \\ \Sigma &= \{s\} \\ \delta(q, a, p) &= \begin{cases} 1 & \text{if } (q, p) \in E, a \in \Sigma \\ 0 & \text{otherwise} \end{cases} \\ I(q) &= 1 \\ F(q) &= \begin{cases} 1 & \text{if } q = v \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

The specificity of this kind of $\mathbb{Q}$-automaton derivable from a graph will become relevant for the optimization of the algorithm developed in the following sections. Note that the alphabet only contains one symbol. However, first we will proceed with the second part of the reduction chain.

Theorem 15. EQUALWALKCOUNT $\leq_l$ $\mathbb{Q}$-EQUIVALENCE

Proof. Let $G = (V, E)$ denote a directed graph where V is the set of vertices and E the set of edges. Let $\mathcal{A}_G^v$ denote a $\mathbb{Q}$-automaton as in Definition 14. Blümel et al. [5] show that the function

$$(G, v, v') \mapsto (\mathcal{A}_G^v, \mathcal{A}_G^{v'})$$

is a valid polynomial reduction for this problem. This reduction is computable by the following log space transducer, with (G, v, v') being the initial string on the input tape. For the sake of readability, the modular description [25] of the log space transducer will be given instead of the state diagram.

1. Write the binary number $c = |V|$ on the work tape. This occupies $\log |V|$ space.
2. Compute $(\mathcal{A}_G^v)$ from (G, v, v').
   a) The vertices V from G are written on the output tape as the states V of $\mathcal{A}_G^v$. No use of the work tape is required.
   b) $\Sigma = \{s\}$ is written on the output tape. No use of the work tape is required, since Σ is a constant and independent from the input.
   c) For δ create a linearized $|V| \times |V|$ matrix on the output tape.
      i. Initialize two binary pointers x and y on the work tape with 1.
      ii. Compare y with c. If $y > c$, increment x by 1 and set y back to 1.

iii. Compare x with c. If $x > c$, proceed with step d).

iv. Check if $(x, y) \in E$. If so, write 1 on the output tape, else write 0. Increment y by 1, and return to step ii).

d) Write $|V|$ times 1 on the output tape as the initial states I. Use of the work tape is not required.

e) For the accepting states F, initialize a binary counter d on the work tape with 1.

f) Compare c with d. If $d > c$, proceed to the next step. Else, if $d = v$ write 1 on the output tape, else write 0 on the output tape. Increment d by 1. Repeat this step.

3. Compute $(\mathcal{A}_G^{v'})$ from (G, v, v') in the same way.

The three binary pointers x, y and d have a maximum length of $\log |V|$, as well as the binary number of vertices c. The total space requirement on the work tape is $4 \log |V| + z$ with z being a constant number of separator symbols between c, d, x and y. All together the space requirement is bound by $O(\log |V|) = O(\log n)$ with $n = |V| + |E|$. Therefore, the reduction is computable in log space. □

Proposition 16 (Log space reducibility)**.** *If $A \leq_l B$ and $B \in NC$, then $A \in NC$ [27].*

With Proposition 16 and the log space reduction chain concluded, the correctness of the initial statement follows.

Theorem 17. EQUIVDIS $\in$ *NC.*

Proof. The correctness follows directly from Theorem 13, Theorem 15, Proposition 8 and Proposition 16. □

3.2 Algorithm to solve EQUIVDIS in RNC

In the previous section the reduction from EQUIVDIS to $\mathbb{Q}$-EQUIVALENCE was discussed. Using these reduction steps, an algorithm to solve EQUIVDIS in RNC can be mapped out. As mentioned above, Kiefer et al.'s [17] algorithm to solve $\mathbb{Q}$-EQUIVALENCE reduces this problem to the zeroness problem for $\mathbb{Q}$-automaton, by combining two $\mathbb{Q}$-automata $\mathcal{A}_1$ and $\mathcal{A}_2$ to an automaton $\mathcal{A}\_$ as described in Definition 18, and subsequently solving the zeroness problem for $\mathcal{A}\_$.

Definition 18 (The automaton $\mathcal{A}\_$ [16])**.** Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two $\mathbb{Q}$-automata. The combined $\mathbb{Q}$-automaton is defined as follows:

$$\mathcal{A}\_ = \{\Sigma, M, \alpha, \eta\} \text{ with}$$

Σ : the alphabet

M : the transition matrix with $M = \begin{pmatrix} M_1 & 0 \\ 0 & M_2 \end{pmatrix}$

α : the vector of initial states with $\alpha = (\alpha_1, -\alpha_2)$

η : the vector of accepting states with $\eta = \begin{pmatrix} \eta_1 \\ \eta_2 \end{pmatrix}$

The algorithm to solve EQUIVDIS therefore consists of three main modules: (i) the conversion of the AAF to a directed graph, (ii) the conversion of the directed graph to a $\mathbb{Q}$-automaton, and (iii) the zeroness test for the combined automaton. In the actual implementation, the conversion step from an AAF to a directed graph can be omitted as it is the identity function. Therefore, it will not be discussed further.

Algorithm 1 Algorithm for solving EQUIVDIS in NC

Input: AAF $F = (A, R)$, arguments $a, b \in A$
Output: **True** if $a \simeq b$, otherwise **false**

```
1: G ← convertToGraph(F)
2: A ← convertToQAutomaton(G)
3: result ← testZeroness(A)
4: return result
```

Algorithm 2 carries out the conversion of a directed graph to a $\mathbb{Q}$-automaton. Instead of generating two automata $(\mathcal{A}_G^v)$ and $(\mathcal{A}_G^{v'})$ as in [5], and subsequently combining them to $\mathcal{A}\_$ as in [17], Algorithm 2 executes both steps and returns the already combined automaton $\mathcal{A}\_$.

Kiefer at al. [17] outline Algorithm 3 to solve the zeroness problem for $\mathbb{Q}$-automata. They state that this algorithm is in RNC "as iterated products of matrices can be computed in NC". Note that the algorithm is not deterministic, but instead probabilistic, using a random vector r in each iteration. The elements of r are selected from the realm of integers from 1 to $K \cdot n$. Following the Schwartz-Zippel lemma[6], K determines the probability of the algorithm returning a correct positive result, i.e., the algorithm correctly returns the zeroness of the combined $\mathbb{Q}$-automaton $\mathcal{A}\_$. The construction of the input automaton in Algorithm 2 can be performed in NC, as the entries of the matrices and vectors can be computed independently in parallel. Consequently and with Definition 9, this preprocessing does not affect the RNC classification of the overall randomized algorithm.

Crucially, Kiefer et al.'s algorithm is able to solve the zeroness problems for all types of $\mathbb{Q}$-automata and not only the specific $\mathbb{Q}$-automaton obtained from the EQUIVDIS reduction containing only one symbol in its alphabet. The relevant difference

[6]The content and proof of this lemma is beyond the scope of this thesis, and can be found in [26, 31].

Algorithm 2 Algorithm for converting an AAF to a $\mathbb{Q}$-automaton

Input: AAF $F = (A, R)$, arguments $a, b \in A$
Output: $\mathcal{A} \leftarrow (n, \Sigma, M, \alpha, \eta)$

$n \leftarrow |A|$
$\Sigma \leftarrow$ "s"
$M \leftarrow (2n \times 2n)$-dimensional matrix
$\alpha \leftarrow 2n$-dimensional vector
$\eta \leftarrow 2n$-dimensional vector
for $i \leftarrow 1$ to n **do**
    for $j \leftarrow 1$ to n **do**
        if $(i, j) \in R$ **then**
            $M[i][j] \leftarrow 1$ # filling the M_1 block
            $M[i+n][j+n] \leftarrow 1$ # filling the M_2 block
        else
            $M[i][j] \leftarrow 0$ # filling the M_1 block
            $M[i+n][j+n] \leftarrow 0$ # filling the M_2 block
    $\alpha[i] \leftarrow 1$
    $\alpha[i+n] \leftarrow -1$
    if $i = a$ **then**
        $\eta[i] \leftarrow 1$
        $\eta[i+n] \leftarrow 0$
    else if $i = b$ **then**
        $\eta[i] \leftarrow 0$
        $\eta[i+n] \leftarrow 1$
    else
        $\eta[i] \leftarrow 0$
        $\eta[i+n] \leftarrow 0$
$\mathcal{A} \leftarrow (2n, \Sigma, M, \alpha, \eta)$
return $\mathcal{A}$

is that for generic $\mathbb{Q}$-automata, M is not one single transition matrix, but a set of $|\Sigma|$ matrices. This becomes relevant in lines 5 and 6 of Algorithm 3. Observing this restriction in the input automata in the case of EQUIVDIS allows to implement a series of simplifications and optimizations, which will be discussed in the following section.

Algorithm 3 Algorithm for testing zeroness [17]

Input: $\mathcal{A} = (n, \Sigma, M, \alpha, \eta)$
Output: **True** if the automaton is zero with confidence $\frac{K-1}{K}$, otherwise **false**
1: **if** $\alpha\eta \neq 0$ **then**
2: **return false**
3: $v \leftarrow \eta$
4: **for** $i = 1$ to n **do**
5: choose a random vector $r \in \{1, 2, ..., Kn\}^{\Sigma}$
6: $v \leftarrow \sum_{\sigma\in\Sigma} r(\sigma)M(\sigma)v$
7: **if** $\alpha v \neq 0$ **then**
8: **return false**
9: **return true**

3.3 Optimized algorithm for solving EQUIVDIS in NC

Algorithm 3 allows to test any $\mathbb{Q}$-automaton for zeroness. However, the input automaton in the case of EQUIVDIS has a very specific structure, permitting a simplification and optimization both in terms of time and space complexity. The first difference is that the input is not any kind of $\mathbb{Q}$-automaton, but an $\mathcal{A}\_$-automaton as described in Definition 18, with the following elements.

$$M = \begin{pmatrix} M_1 & 0 \\ 0 & M_2 \end{pmatrix}$$

$$\alpha = (\alpha_1, -\alpha_2)$$

$$\eta = \begin{pmatrix} \eta_1 \\ \eta_2 \end{pmatrix}$$

When considering the $\mathbb{Q}$-automaton derived from the reduction from EQUIVDIS, the resulting automaton $\mathcal{A}\_$ is even more restricted. In the following overview of restrictions, the index $_{ED}$ will denote the automaton obtained by reducing from EQUIVDIS.

1. α_{ED} is a bi-partite row vector populated with 1 in the first half, and -1 in the second half.

2. η_{ED} is a column vector populated by 0 except for two positions where they are 1 (corresponding to the two input arguments), with the first being in the first half and the second in the second half of the vector.

3. Σ_{ED} contains only one symbol.
4. M_{ED} contains only one matrix (this follows immediately from restriction 3).
5. M_{ED} is a block matrix of the form $\begin{pmatrix} M' & 0 \\ 0 & M' \end{pmatrix}$ with M' being derived from the attack relation of the input AAF and translated to the adjacency matrix of the corresponding directed graph.

Based on these restrictions, Algorithm 3 can be simplified in the following ways.

a) Eliminate the if statement in lines 1-2.

Proof. From restriction 2 it follows that the vector η only contains two 1 at index a and b. The vector product $\alpha\eta$ is therefore the sum of elements of α at positions a and b. From restriction 2 we know that a is in the first half of the vector and b in the second half. From restriction 1 we know that the vector α is populated by 1 in the first half, and -1 in the second half.
For these reasons $\alpha\eta$ always yields $1 + (-1) = 0$. The if condition therefore always evaluates to false, and can therefore be omitted. □

b) Instead of using a random vector r in line 5, use a random integer.

Proof. From restriction 3 we know that Σ contains only one symbol. Therefore, r_{ED} is a one-dimensional vector and can thus be considered a randomized integer scalar. □

c) Eliminate the sum from line 6.

Proof. From restriction 3 we know that Σ contains only one symbol. From restriction 4 we know that M contains only one matrix. From b) we know that r is an integer. The sum therefore only contains one element and can thus be simplified to rMv. □

d) Split the matrix-vector multiplication $v' = Mv$ in line 6 in two parts $v'_1 = M'v_1$ and $v'_2 = M'v_2$.

Proof. In the matrix-vector multiplication $v' = Mv = \begin{pmatrix} M' & 0 \\ 0 & M' \end{pmatrix} v$, when calculating the upper half of v' only the upper half of v contributes to the sum $\sum_{i=1}^{n} M_i v_i$, since the lower half of v is matched with the zeros from upper right quadrant of M. The inverse applies to calculating the lower half of v'. □

e) Substitute the vector multiplication $\alpha\eta$ and zero comparison in line 7 by an equality test of two sums $\sum_{i=1}^{\frac{n}{2}} v'_1[i]$ and $\sum_{j=1}^{\frac{n}{2}} v'_2[j]$.

Proof. From restriction 1 we know that α is a bipartite vector with 1 in the first half, and -1 in the second half. The vector v contains only positive integers,

as the input adjacency matrix M contains only 1 and 0, v is initialized with η which also contains only 1 and 0, and is multiplied by a random positive integer r. The vector multiplication $\alpha\eta$ therefore yields the sum of $\frac{n}{2}$ positive integers and $\frac{n}{2}$ negative integers. The sum is zero if the absolute values of the first half and the second half sum to the same value. Thus, the condition $\alpha\eta \neq 0$ is equivalent to the vector sum of v_1' and v_2' not being equal. □

f) Use AAF F as input instead of $\mathbb{Q}$-automaton $\mathcal{A}\_$.

Proof. From the simplifications above, it follows immediately that the new algorithm no longer requires all elements of the automaton $\mathcal{A}\_ = (n, \Sigma, M, \alpha, \eta)$. The only necessary elements are the number of states n, the upper left quadrant of M (i.e., the original adjacency matrix of the AAF) and η. The number of states n is two times the number of arguments in the initial AAF. η is a column vector of length $2|A|$ with 1 in the position a and $b + |A|$, with a, b being the arguments from the original AAF to be tested for equivalence. □

g) Remove the random integer r (see step b)) in line 5.

Proof. In step b) it was proven that r is a random integer. In each iteration a new random integer is generated and multiplied as a scalar to $M'v_i$ with $i \in \{1, 2\}$. This scalar only introduces a common positive multiplicative factor. Since the algorithm compares only the equality of the column sums, this factor cancels out and therefore does not affect the result.
After x iterations the following holds:

$$v_i^{(x)} = \left(\prod_{j=1}^{x} r_j\right) M v_i^{(x-1)}$$

.

Therefore, the following holds for the comparison of the column sums:

$$\sum_{i=1}^{n} v_1^{(x)}[i] \stackrel{?}{=} \sum_{i=1}^{n} v_2^{(x)}[i]$$

$$\Leftrightarrow \sum_{i=1}^{n} \left(\prod_{j=1}^{x} r_j\right) M v_1^{(x-1)}[i] \stackrel{?}{=} \sum_{i=1}^{n} \left(\prod_{j=1}^{x} r_j\right) M v_2^{(x-1)}[i]$$

$$\Leftrightarrow \left(\prod_{j=1}^{x} r_j\right) \sum_{i=1}^{n} M v_1^{(x-1)}[i] \stackrel{?}{=} \left(\prod_{j=1}^{x} r_j\right) \sum_{i=1}^{n} M v_2^{(x-1)}[i]$$

$$\Leftrightarrow \sum_{i=1}^{n} M v_1^{(x-1)}[i] \stackrel{?}{=} \sum_{i=1}^{n} M v_2^{(x-1)}[i]$$

The last line is achieved by simply dividing both sides by the cumulative product of the scalars r_j. □

These simplifications yield Algorithm 4 for solving EQUIVDIS using the zeroness test in RNC by Kiefer et al. [17] as basis. Crucially, by removing the randomization element in g), the resulting algorithm becomes deterministic. Let us consider the different components of this algorithm. As discussed in the previous section, the preprocessing can be carried out in NC. The main part, which has now become deterministic, consists of iterated matrix-vector multiplications, iterated additions for the column sums, and the integer comparison. Since all these operations can be performed in NC [13,17,23], the resulting Algorithm 4 is in NC.

Algorithm 4 Optimized algorithm for solving EQUIVDIS

Input: AAF $F = (A, R)$, arguments $a, b \in A$
Output: **True** if $a \simeq b$, otherwise **false**

```
1: M ← AdjacencyMatrix(F)
2: v1, v2 ← |A|-dimensional vectors initialized with 0
3: v1[a] ← 1
4: v2[b] ← 1
5: for i = 1 to 2|A| do
6:     v1 ← M v1
7:     v2 ← M v2
8:     if Σ_{i=1}^{|A|} v1[i] ≠ Σ_{j=1}^{|A|} v2[j] then
9:         return false
10: return true
```

The benefits of these simplifications and optimizations regard both the time and space complexity of the algorithm. While dispensing with Σ and α has a relatively minor impact on the complexity, the use of the adjacency matrix of the AAF instead of the one of $\mathcal{A}\_$ is noteworthy. A matrix occupies $O(n^2)$ space; reducing the dimension from $n \times n$ to $\frac{n}{2} \times \frac{n}{2}$ therefore only requires a quarter of the original space. The same applies to time; the matrix-vector multiplication Mv has a $O(n^2)$ time complexity. Halving the dimensions of M and v and considering v_1 and v_2 separately therefore only requires $O(2(\frac{n}{2})^2) = O(\frac{1}{2}n^2)$. Furthermore, the costly matrix operations of Algorithm 2 to convert the AAF to a $\mathbb{Q}$-automaton become unnecessary.

3.4 Algorithm to solve STRONGERDIS in NC

In the previous sections, an algorithm was developed to test two arguments for equivalence. However, unstructured preliminary tests revealed that the equivalence of two arguments is very rare. Testing randomly generated AAFs ranging from 10 to 1000 arguments (see Section 4.2 on input generation) and approximately 500 tests for each input size, the algorithm only returned true (i.e., that the two tested arguments were equivalent) in less than 5% of the instances. In these cases,

it remains unclear which of the two arguments is stronger. Furthermore, from a theoretical point of view STRONGERDIS is the more powerful and useful problem to solve. EQUIVDIS can be solved by combining the results of STRONGERDIS(a,b) and STRONGERDIS(b,a), i.e., if both return true, $a \succeq b$ and $b \succeq a$ hold, and therefore $a \simeq b$ holds. For these reasons, the development of the NC Algorithm 5 to solve STRONGERDIS seems relevant.

Algorithm 5 NC algorithm for solving STRONGERDIS

Input: AAF $F = (A, R)$, arguments $a, b \in A$
Output: **True** if $a \succeq b$, otherwise **false**

$M \leftarrow$ AdjacencyMatrix(F)
$v_1, v_2 \leftarrow$ array of length $|A|$ initialized with 0
for $i \leftarrow 1$ to $|A|$ **do**
    if $i = a$ **then**
        $v_1[a] \leftarrow 1$
    if $i = b$ **then**
        $v_2[b] \leftarrow 1$
for $i = 1$ to $2|A|$ **do**
    $v_1 \leftarrow Mv_1$
    $v_2 \leftarrow Mv_2$
    if i is odd **then**
        if $\sum_{i=1}^{|A|} v_1[i] > \sum_{j=1}^{|A|} v_2[j]$ **then**
            return false
        else
            return true
    if i is even **then**
        if $\sum_{i=1}^{|A|} v_1[i] > \sum_{j=1}^{|A|} v_2[j]$ **then**
            return true
        else
            return false
return false

Algorithm 5 is based on the optimized algorithm to solve EQUIVDIS, with calculations being identical. The crucial difference are the conditions yielding the return value. However, the question is how the correctness of this algorithm can be proven. Similarities to the STRONGERDIS algorithm proposed by Blümel et al. [5] are visible; both algorithms distinguish between even- and odd-length walks and consider the column sums of the vectors corresponding to argument a and b. In Algorithm 5, the crucial matrix operation is $v_1 \leftarrow Mv_1$. With v_1 being initialized with η_a, in every iteration i the product Mv_1 returns exactly column a of M^i. The broad idea of the mathematical proof is outlined below.

Theorem 19. *For all iterations i the column v_x in the NC algorithms is equal to the x-th column of the matrix M^i of Blümel et al.'s algorithms.*

Proof. The proof proceeds by mathematical induction. Let $F = (A, R)$ be an AAF and $a \in A$. Let η_a denote the a-th standard basis vector and define

$$v_0 = \eta_a, \qquad v_n = Mv_{n-1} \quad (n \geq 1).$$

We show that v_n is the a-th column of M^n.
Base case. Since

$$v_1 = M\eta_a,$$

and multiplication by the standard basis vector η_a selects the a-th column of a matrix, v_1 is precisely the a-th column of $M = M^1$.
Induction hypothesis. Assume that v_n is the a-th column of M^n.
Induction step. Then

$$v_{n+1} = Mv_n.$$

By the induction hypothesis, v_n equals to the a-th column of M^n. Left-multiplication by M therefore yields the a-th column of

$$MM^n = M^{n+1}.$$

Hence, v_{n+1} is the a-th column of M^{n+1}.
Therefore, the claim holds for all $n \geq 1$. □

The random integer r was already removed in the optimization of the EQUIVDIS algorithm in the previous section. The proof outlined above furthermore shows that fundamentally both Blümel et al.'s algorithm and the NC algorithm developed in this thesis use the concept of walks and discussion counts, with the difference being that the former computes the discussion counts for all arguments, while the latter only computes the discussion count for the two arguments in question. This allows to lower the threshold for the optimized algorithms to $2|A| - 1$ as proven for Blümel et al.'s algorithm, as opposed to $2|A|$ in Kiefer et al.'s original algorithm.

The similarity of the two algorithms is interesting as it shows that both follow the same principle and might be useful in different circumstances. Blümel et al.'s algorithm presents with a higher time complexity, due to its matrix-matrix multiplications ($O(n^3)$), however, once concluded it allows to establish the complete ranking on the AAF. The NC algorithm is more time efficient as it only uses matrix-vector multiplications ($O(n^2)$), but allows only to compare two arguments at a time.

3.5 Run time implications

As previously discussed, problems in the NC complexity class can be parallelized efficiently. The NC algorithms outlined in the previous sections use iterated matrix-vector multiplications $v \leftarrow Mv$. The computation of each line of v can be executed independently of other lines, thus lending itself to parallelization by computing each line in a different thread. However, the parallelization process itself requires resources for tasks such as input chunking, thread handling and fork joining. The

question whether parallelization is beneficial to the total run time in the cases of StrongerDis and EquivDis therefore remains open, and may well depend on variables such as the input size or the number of available CPU cores.

Blümel et al. [5] present an algorithm to solve StrongerDis which uses matrix-matrix multiplications. Similarly to the matrix-vector operation in the algorithms developed in this thesis, their algorithm may benefit from parallelization as well. In this case, the parallelization can occur on row level as discussed above or the level of each individual cell. While Blümel et al. never claim that their algorithm is in NC, at this point, the question arises whether it might. As answering this question lies beyond the scope of this thesis, the NC attribute will no longer be used to reference to the optimized algorithm developed in the previous sections. In the continuation of this work, Blümel et al.'s algorithm will be called **MM algorithm** for its basis in matrix-matrix multiplication. The algorithm developed in this thesis will be called **MV algorithm** for its basis in matrix-vector multiplication.

Independently from the possible benefits of parallelization, it may be interesting to compare the run time complexity of the MM and MV algorithms. At first glance the answer seems obvious, as the worst-case time complexity of the former is $O(n^4)$ ($O(n^3)$ for matrix-matrix multiplication times n for the for-loop), whereas the latter's is $O(n^3)$ ($O(n^2)$ for matrix-vector multiplication times n for the for-loop). The operation yielding the worst-case complexity is the iterated matrix-matrix multiplication in case of the former, and the matrix-vector multiplication in case of the latter. However, the average complexity might well be different for both cases. A difference can also be expected between the two problems StrongerDis and EquivDis, as well as based on the result. E.g. a negative result for EquivDis can be found in the first iteration, whereas a positive result requires the completion of all iterations.

The empirical tests of the next section aim to answer some of the questions raised in this section.

4 Methodology and Test Design

In this section, the test design, as well as the input and the hardware used for the experiments will be discussed. Furthermore, the research questions establish which parameters will be considered in the experiments. The section concludes with the implementation details, and the validation of the developed implementations.

4.1 Research questions

In this section, the questions to be answered by the empirical test will be discussed. The run time will likely be influenced by the following (and perhaps other) parameters.

- Problem: EQUIVDIS, STRONGERDIS
- Algorithm: MM, MV
- Implementation: sequential, parallel, space-optimized
- Input size: number of arguments in the AAF
- Input density: attack probability in the AAF
- Graph structure: e.g., number of strongly connected components (SCCs), cyclicity, depth, width
- Result: true or false

The goal is to test the time performance, and analyze the effect of the variables algorithm, implementation and input size. The remainder of the parameters will only be mentioned briefly. The problem is usually given and immutable. Therefore, for both the EQUIVDIS and the STRONGERDIS problem, the following questions will be considered individually. Including more parameters would increase the experiment size beyond the scope of this bachelor's thesis. However, the possible relevance of these parameters needs to be kept in mind when interpreting the results.

Parallelization

The first step is to find the optimal implementation for each algorithm, taking into account the input size. Parallelization can be used in both the MM and the MV algorithms to compute matrix operations. In the MM algorithm iterated matrix-matrix multiplications are calculated; in the MV algorithm matrix-vector multiplications are calculated. In both cases, each line of the output matrix/vector can be computed independently, and hence parallelized. The hypothesis is that there is a threshold which determines the break-even point after which the benefits of parallelization outweigh the thread-management overhead. This threshold might depend on the input size and other parameters. Making use of this potential threshold, the optimal

version of the MM and the MV algorithms for both EQUIVDIS and STRONGERDIS will be implemented.

Space optimization

Unstructured preliminary tests revealed that the restricting resource in many cases is not time but space. While computation time for all algorithms and implementations remained within the range of a few seconds in the great majority of test runs, the Java Virtual Machine (JVM) running on a MacBook (Apple M2 chip, 8 CPU cores, OS Tahoe 26.3.1, 16GB RAM) only tolerated input sizes of AAF with $|A| < 8000$ (attack probability of 0.5). Above this threshold the generation of the input adjacency matrix as two-dimensional `double` array caused an OutOfMemory exception before the algorithms were even executed.

While the MM algorithms compute M^i, the need to store the full matrix can hardly be avoided. The MV algorithms, on the other hand, only require the initial adjacency matrix M for the matrix-vector multiplication. As M is only populated by 1 and 0, instead of using a matrix, the HashMap used in the `DungTheory` class of the Tweety Project library[7] can be used to develop a more space-efficient version of the MV algorithms. This just-in-time version of the MV algorithm does not use an adjacency matrix as input but calls the `isAttackedBy(b, a)` method of the `DungTheory` object as computation occurs. However, it is to be expected that this space efficiency comes at the cost of worse time performance. In case of the MV algorithms the performance of this space-optimized implementation will be compared to the optimal version of EXP1.1 and EXP2.1.

Comparison of the MM and the MV algorithms

Once the optimal implementation for each algorithm is established, the run time of the MM and the MV algorithms will be compared. For this purpose, instances of the ICCMA competition 2025 [2] will be used instead of randomly generated test instances. The ICCMA instances have been extensively analyzed and benchmarked, and providing run-time results on these instances might allow fruitful comparison with previous work. Furthermore, some of the ICCMA instances are derived from real-life graph structures, such as railway or subway networks.

Experiment overview

EXP1.1: Analysis of how parallelization affects the run time of both algorithms for solving EQUIVDIS.

EXP1.2: Analysis of how a space optimized implementation of the MV algorithm compares to the optimal version from EXP1.1.

[7] See https://tweetyproject.org/

EXP1.3: Analysis of how the run time of the optimal MV algorithm for solving EQUIVDIS compares to the optimal MM algorithm.

EXP2.1: Analysis of how parallelization affects the run time of both algorithms for solving STRONGERDIS.

EXP2.2: Analysis of how a space optimized implementation of the MV algorithm compares to the optimal version from EXP2.1.

EXP2.3: Analysis of how the run time of the optimal MV algorithm for solving STRONGERDIS compares to the optimal MM algorithm.

Figure 5 illustrates the test design for EQUIVDIS; the setup for STRONGERDIS is analogous.

4.2 Input

In this section, the input used for the different experiments will be described. For the experiments comparing different implementations for one algorithm, randomly generated AAFs are used. The experiments comparing the two algorithms take the AF instances of the ICCMA 2025 competition as input.

4.2.1 EXP1.1, EXP1.2, EXP2.1, EXP2.2: DungTheoryGenerator

For generating the test instances for the experiments EXP1.1, EXP1.2, EXP2.1, EXP2.2, the `DungTheoryGenerator` of the Tweety Project Abstract Argumentation library will be used. The parameters of the `DungTheoryGenerator` are the number of arguments and attack probability between two arguments. The method `next()` generates an AAF with the requested parameters. Using the `DungTheoryGenerator` allows to control for specific sizes and density. The input sizes that will be considered are AAFs with 10, 50, 100, 500, 1000, 5000 and 10,000 arguments. 10,000 is close to the upper threshold of memory requirements tolerated by the FernUniversität server; larger inputs cause a heap overflow exception. As for the attack probability (i.e., graph density) the fixed value of 0.5 is used, since considering different input densities in combination with the previously discussed parameters would go beyond the scope of this thesis.

On each AAF test instance, 1000 test runs will be executed. This ensures the generation of a sufficiently large data set to obtain reliable median run times. Due to the fact that generating large AAFs takes a considerable amount of computation time, a threshold of 4999 was chosen for generating a new AAF for each test run. At this threshold the probability of two random arguments not repeating in 1000 test runs becomes roughly 98% (see Lemma 1). Therefore, for the input sizes 10, 50, 100, 500 and 1000, a new AAF will be generated for each test run, whereas for input sizes 5000 and 10,000, only one AAF will be generated. For each test run, two

random arguments from the AAF will be selected using the `ThreadLocalRandom` functionality of the Java standard library.

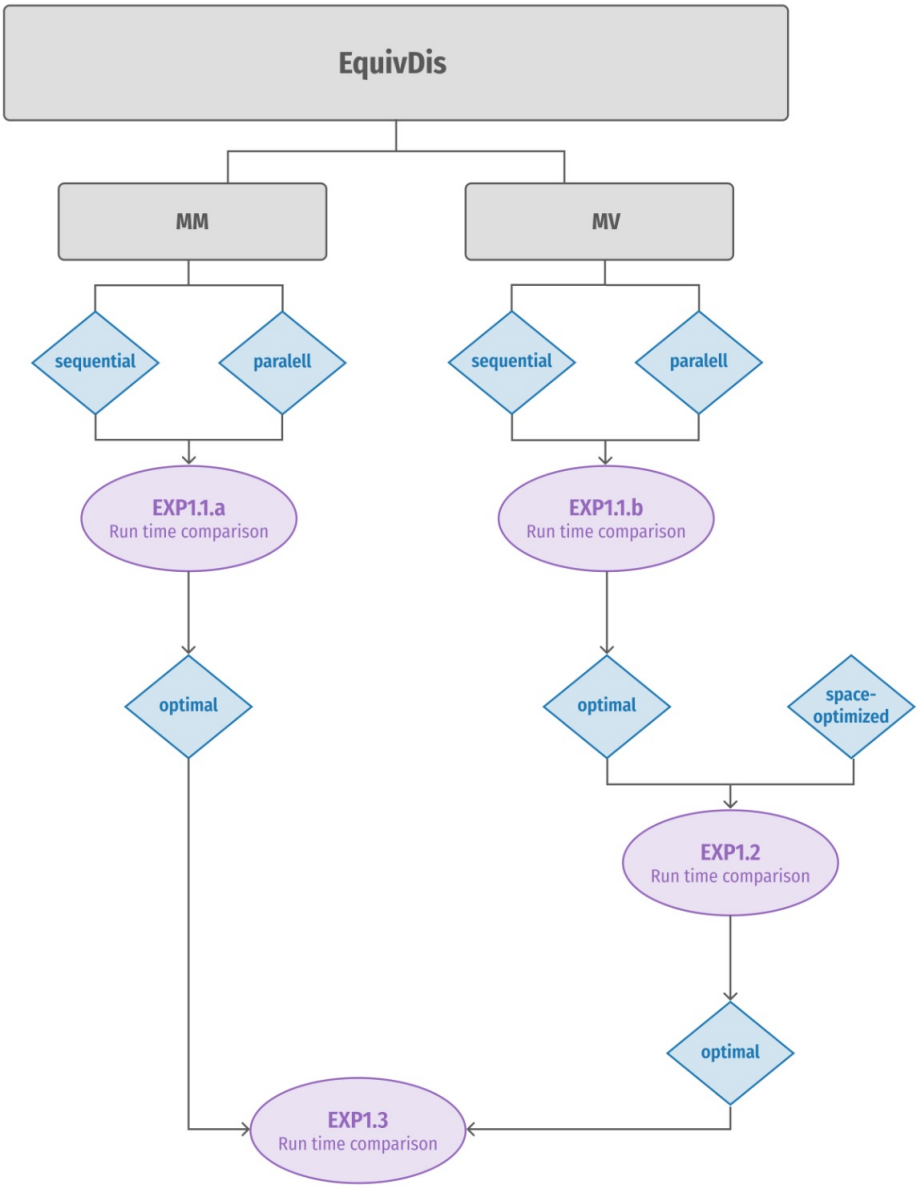

Figure 5: Test design for EQUIVDIS.

Lemma 1. *Let n denote the number of arguments of an AAF and assume that $k = 1000$ ordered pairs of arguments are chosen independently and uniformly at random. The probability that no ordered pair occurs twice is approximately*

$$P_{\text{distinct}} \approx \exp\left(-\frac{k(k-1)}{2n^2}\right).$$

For $n = 5000$, this probability exceeds 95%.

Proof. Since ordered pairs are sampled, there are

$$N = n^2$$

possible outcomes. Choosing k pairs independently corresponds to the classical birthday problem. The probability that no duplicate occurs is approximated by

$$P_{\text{distinct}} \approx \exp\left(-\frac{k(k-1)}{2N}\right).$$

Substituting $N = n^2$ and $k = 1000$ yields

$$P_{\text{distinct}} \approx \exp\left(-\frac{1000 \cdot 999}{2n^2}\right).$$

For $n = 5000$,

$$P_{\text{distinct}} \approx \exp\left(-\frac{999000}{2 \cdot 5000^2}\right) = \exp(-0.01998) \approx 0.980.$$

Hence, the probability that all 1000 sampled ordered pairs are distinct is approximately 98%. Therefore, using frameworks with at least 5000 arguments makes duplicate comparisons highly unlikely. □

4.2.2 EXP1.3, EXP2.3: ICCMA instances

For EXP1.3 and EXP2.3, i.e., the comparison of the optimized MM and MV implementations, the instances from the ICCMA competition 2025 [2] will be used. On the one hand, these instances have been extensively benchmarked in the ICCMA competition, and sometimes contain examples from real-life graphs, such as traffic networks. The generators of the ICCMA instances can be found in [2, 12]. On the other hand, while an exhaustive analysis of the graph structure of these instances is beyond the scope of this work, it might still produce results which can point in interesting directions for future work.

In order to gain an overview of some of the graph properties present in the benchmarked ICCMA instances, a brief analysis of the different categories was carried out, using the JGraphT library[8]. The results are depicted in Table 3. The condensation graph was used to determine the maximum depth and width, as it transforms each SCC into a single vertex and therefore yields a directed acyclic graph. This allows the hierarchical structure of the argumentation framework to be characterized without the ambiguity introduced by cycles in the original graph.

[8] See https://jgrapht.org/.

Category	Nodes	Edges	Density	# SCCs	Largest SCC	Max width	Max depth	Cyclicity
ABA2AFs	118 – 1221	8700 – 1.44813e+06	0.630 – 0.994	2 – 30	108 – 1210	1 – 19	1 – 2	true
AFBenchGen2_BA	101 – 201	111 – 343	0.005 – 0.019	10 – 200	2 – 129	3 – 71	2 – 7	true
AFBenchGen2_ER	101 – 502	1045 – 125492	0.050 – 0.499	1 – 1	101 – 502	1 – 1	0 – 0	true
AFBenchGen2_WS	100 – 500	600 – 8000	0.010 – 0.121	1 – 11	100 – 500	1 – 7	0 – 3	true
AFGen	100 – 512	2025 – 78645	0.164 – 0.309	1 – 1	100 – 512	1 – 1	0 – 0	true
AdmBuster	1000 – 2000	1498 – 2998	0.001 – 0.001	1000 – 2000	1 – 1	2 – 2	500 – 1000	false
Datalog	96 – 1948	1856 – 1.4565e+06	0.204 – 0.511	41 – 460	40 – 1841	40 – 459	1 – 1	true
KWT	41 – 501	1561 – 99644	0.245 – 0.993	2 – 101	40 – 401	2 – 100	0 – 1	true
Planning2AFs	108 – 1387	168 – 2563	0.001 – 0.015	12 – 1237	2 – 421	6 – 234	2 – 54	true
SccGenerator	280 – 1554	3598 – 604639	0.017 – 0.251	2 – 50	22 – 812	1 – 5	0 – 19	true
SemBuster	150 – 1800	2650 – 361800	0.112 – 0.119	100 – 1200	2 – 2	2 – 2	50 – 600	true
StableGenerator	156 – 981	858 – 9725	0.003 – 0.065	7 – 260	131 – 955	3 – 93	2 – 16	true
Traffic	127 – 14812	142 – 28437	0.000 – 0.019	5 – 5653	4 – 8486	2 – 1355	3 – 26	true

Table 3: Graph structure of the ICCMA 2025 instances.

The number of repetitions was limited to 100, as preliminary tests revealed that the IO costs for reading the AAFs from the input files plus the higher computation time for certain ICCMA instances cause EXP1.3 and EXP2.3 to take considerably more time than the previous experiments. Thus, a smaller repetition number allowed to use a greater number of different ICCMA instances. Furthermore, the ICCMA instances were sorted by input size, and the experiment proceeded from smallest to largest input sizes. Due to time restrictions, data collection was concluded on the August 10, 2026. At this point, 220 of the 323 ICCMA 2025 AF instances were used (the ABA instances were excluded). The instances that were benchmarked can be found in Appendix A.

4.3 Measurement

In order to account for Just-In-Time (JIT) compilation and optimizations by the JVM, before each test run a warm-up is executed. This produces more stable and reliable run time measurements [7]. In case of EXP1.1, EXP1.2, EXP2.1 and EXP2.2, the warm-up consists of 1000 repetitions of the tested method on a small, random set of AAFs (10 arguments). In case of EXP1.3 and EXP2.3, the warm-up consists of 10 repetitions per implementation for each ICCMA instance.

In each experiment, two implementations are compared. To allow comparability, for each pair of implementations the run time was measured for both implementations on the same input instance, i.e., AAF and arguments a, b.

The measurement was carried out using the `System.nanoTime()` method of the Java standard library. The use of specialized benchmarking libraries was discarded for several reasons. On the one hand, preliminary tests showed that the run times were not within the range of nanoseconds, which would have required a specific tool to properly evaluate such small run times. On the other hand, libraries such as the Java Microbenchmark Harness [28] execute each test several times (sometims several thousand times) to obtain robust medians for each test. Preliminary tests showed that the experiments took several hours in the best case, and several weeks in the worst case, with each test run for an AAF and two arguments being executed only once. If each test run consisted of 1000 executions, the experiment would have not been feasible in the given time. The comparison of different argument combinations was considered more interesting than the hardware-based run time variance within one test run.

4.4 Hardware

The main experiments were executed on a virtual machine provided by the FernUniversität in Hagen (16 CPU cores, 64GB RAM, OS Ubuntu). The experiments were started using the command `java -Xms60g -Xmx60g -jar experiment.jar` allotting a heap size of 60GB to the experiment.

```
SimpleMatrix Mnew =  M.mult(F);
SimpleMatrix vNew = F.mult(v);
```

Figure 6: Parallelization for MV algorithms using EJML.

4.5 Implementation

In this section, different Java implementations for the MM and MV algorithms will be discussed. For both MV algorithms, three versions were implemented. All implementations are available on GitHub[9].

1. Sequential, using the Efficient Java Matrix Library (EJML)[10] for matrix operations
2. Parallelized, using Java standard library
3. Space optimized, using the `isAttackedBy(b, a)` method of the Tweety Project library

For the MM algorithms, two versions were implemented, as the space-optimized version is not compatible with the MM algorithms.

1. Sequential, using EJML library for matrix operations
2. Parallelized, using Java standard library

4.5.1 Sequential implementation using Efficient Java Matrix Library

For the sequential implementation, the EJML was used. EJML is a library used for algebraic operations, and is especially designed to optimize matrix operations. The use of EJML avoids the problem of inefficient and error-prone manual implementations. Furthermore, the use of EJML allows for clean and readable code. Figure 6 illustrates the implementation of matrix-matrix and matrix-vector multiplications. While many other linear algebra libraries exist, EJML additionally provides simple syntax for common matrix operations such as multiplications via the SimpleMatrix package.

4.5.2 Parallel implementation using Java standard library

Parallelization was implemented using Java's parallel stream API, which internally relies on the Fork/Join framework to automatically distribute independent tasks across multiple cores [22]. This approach enables efficient utilization of modern multi-core architectures while avoiding manual thread management [15]. Parallel streams are particularly suitable for CPU-bound computations on large, easily

[9] See github.com/vanessakoebke/BachelorThesis
[10] See ejml.org

```
double[] v1New = new double[n];
double[] v2New = new double[n];
IntStream.range(0, n).parallel().forEach(i -> {
    double sum1 = 0;
    double sum2 = 0;
    for (int j = 0; j < n; j++) {
            sum1 += F[i][j] * v1[j];
            sum2 += F[i][j] * v2[j];
    }
    v1New[i] = sum1 * r;
    v2New[i] = sum2 * r;
});
```

Figure 7: Parallelization for MV algorithms using Java standard library.

```
double[][] Mnew = new double[n][n];
IntStream.range(0, n).parallel().forEach(i -> {
     for (int j = 0; j < n; j++) {
           double sum = 0;
           for (int k = 0; k < n; k++) {
                 sum += M[i][k] * F[k][j];
           }
          Mnew[i][j] = sum;
     }
});
```

Figure 8: Parallelization for the MM algorithms using Java standard library.

partitionable data structures, such as matrix operations [24]. The implementation of this thesis uses `IntStream.range().parallel()` as intermediate operation and `forEach()` as terminal operation [21].

Figures 7 and 8 illustrate the key components of the parallel implementation. In order to parallelize computation over the index space of the input array (instead of the elements of the array), the static method `range(0, n)` is used. This generates the set of indizes of the array. The `parallel()` method then converts the `IntStream` into a parallel stream. The terminal operation `forEach()` allows to process each row of the matrix/each line of the vector independently.

4.6 Algorithm Validation

In order to validate the different implementations, they were tested for correctness using repeated JUnit tests. The validation occurs in two phases. First, the sequential MM algorithms for EQUIVDIS and STRONGERDIS as outlined in Blümel et al. [5] were tested against the comparative methods of the Tweety Project library's discussion-based semantics framework, to ensure the correct implementation of

their algorithm.

At the moment of writing this thesis, the `DiscussionBasedRankingReasoner` class of the Tweety Project version 1.30 is using 6 as the upper limit until which walks are considered. The novel work of Blümel et al. proved that a valid limit is $2|A| - 1$. Before executing the validation, the upper limit was corrected in the `DiscussionBasedRankingReasoner` class. A pull request to adopt this change will be created upon completion of this thesis.

After the correctness of the sequential MM algorithms was established, both the parallelized implementations of the MM algorithms, as well as the different implementations of the MV algorithms was tested against the sequential MM benchmark.

Each set of tests was repeated 1000 times.

5 Results and Discussion

In this section, the results of the different experiments will be presented and discussed. The results for StrongerDis and EquivDis are very similar in nature. For the sake of readability, only the graphs for the StrongerDis experiments will be shown in this section. The graphs of all experiments can be found in Appendices B and C. The section concludes with the limits of validity to the presented results.

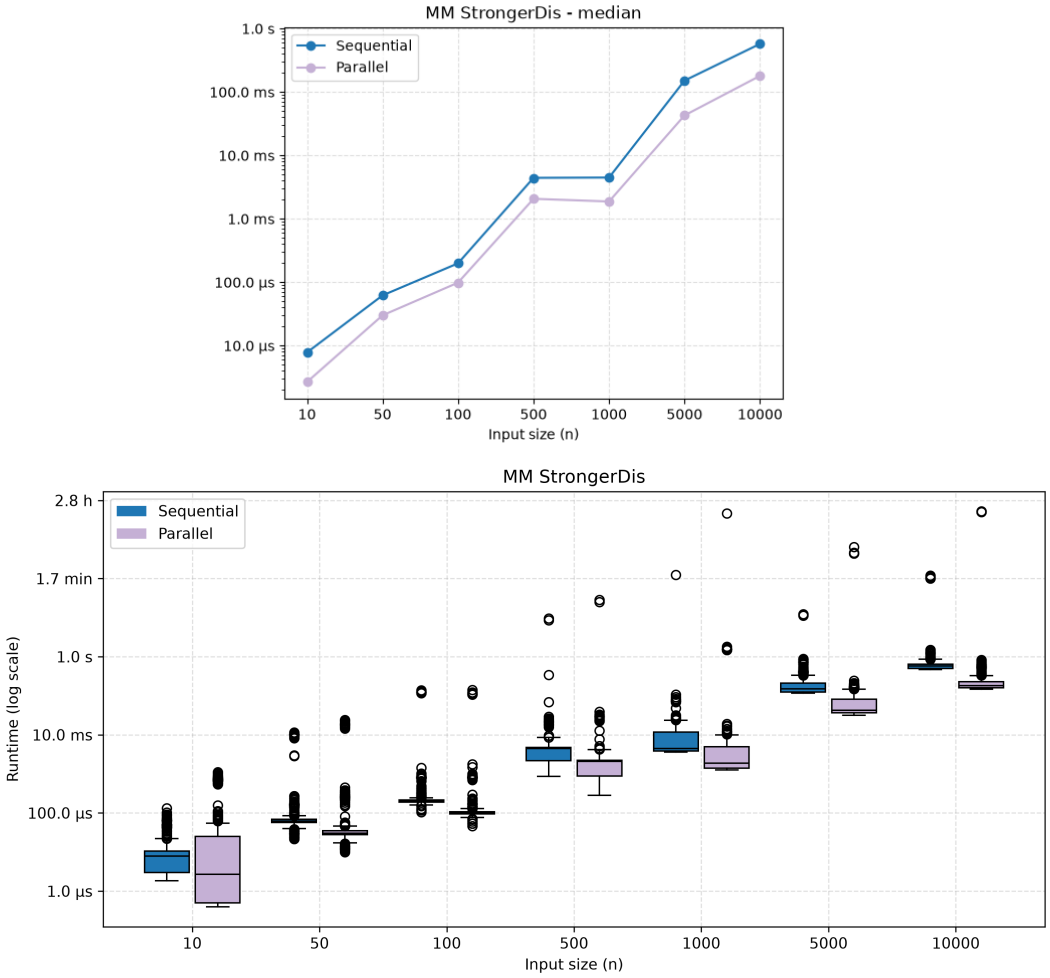

Figure 9: Comparison of the sequential and parallel implementations of the MM algorithm for solving StrongerDis.

5.1 EXP1.1 and EXP2.1: Effects of parallelization

Figure 9 depicts the results for the MM StrongerDis algorithm, while Figure 10 depicts the results for the corresponding MV algorithm. The upper graphs show the median per input size and implementation. The bottom graphs show the distribution of the 1000 test instances per input size and implementation. While the median run time for all implementations remains well under 1 second, there are several outliers that severely deviate from the median by at least one order of magnitude. The most extreme example can be found in the parallelized MM implementation, where most instances terminate within 1 second, however, one single outlier required roughly 3 hours of computation.

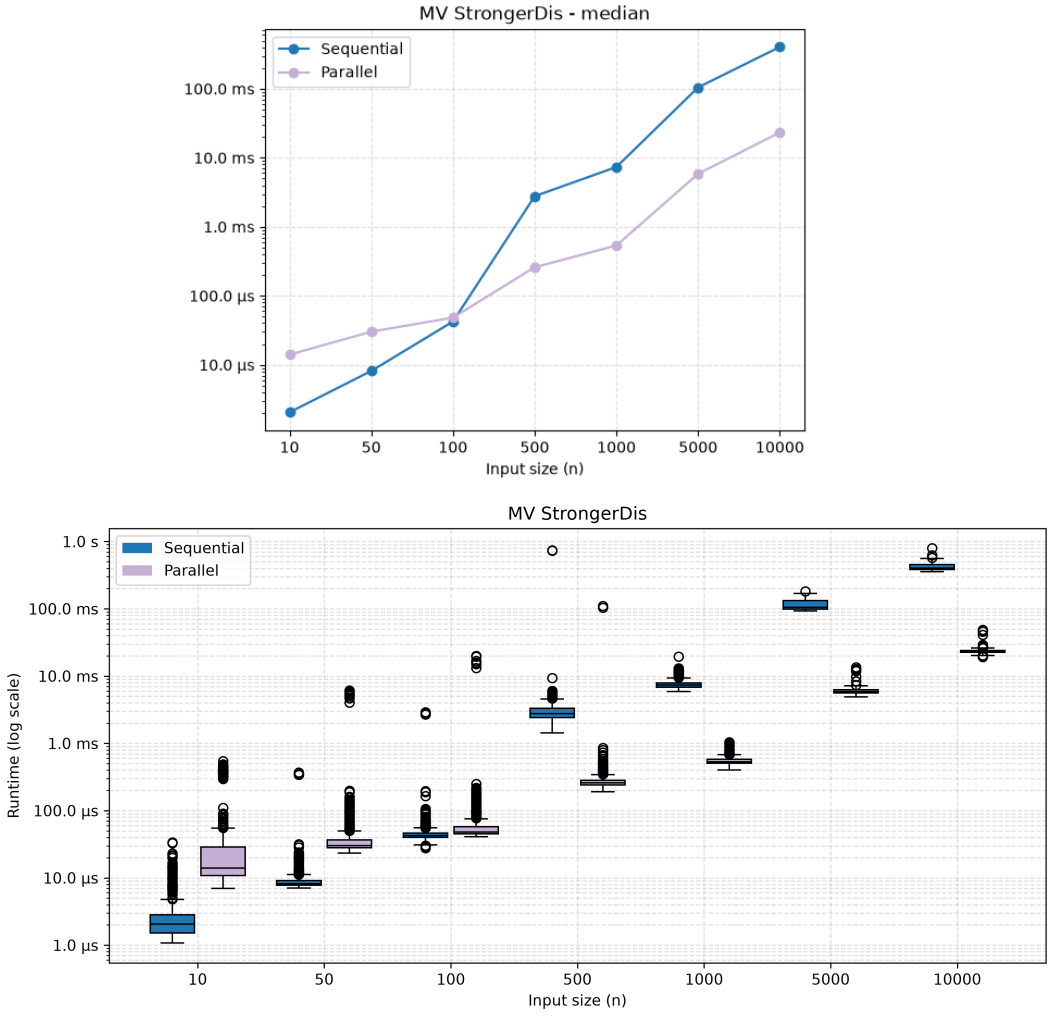

Figure 10: Comparison of the sequential and parallel implementations of the MV algorithm for solving StrongerDis.

In the case of the MV algorithms, as was expected, parallelization benefits the run time for larger input sizes. For small input sizes, the parallelization overhead (in this case mostly thread handling and fork joining) outweighs the speed-up. The break-even point is at approximately 100 arguments per AAF for both MV algorithms [11]. In the case of the MM algorithms, on the other hand, the parallelized implementation outperforms the sequential implementation almost from the smallest input size of 10 arguments. This is probably due to the fact that matrix-matrix multiplication in itself is more costly ($O(n^3)$) than the matrix-vector multiplications ($O(n^2)$) of the MV algorithms.

Curiously, while the parallel implementation outperforms the sequential implementation for large input sizes, in the case of outliers whose computation takes more than 1 second, the parallel implementation consistently performs worse in the MM algorithms and better in the MV algorithms. Finally, parallelization benefits the MV algorithm more strongly (by about an order of magnitude starting at input size 500) than the MM algorithm (improvement by less than half an order of magnitude). These are surprising findings whose explanation requires further research.

For EXP1.2 and EXP2.2, the MV algorithms were optimized by introducing a condition on the input size. If the input is smaller than 100 the sequential implementation is used, else the parallel implementation. The MM implementations are not altered, as there is no significant benefit to the sequential implementation, except for extreme outliers. These are, however, very rare, and do not seem to justify a worsened run time in the great majority of instances. The parallel implementation was thus selected as the optimal version for the experiment of EXP1.3 and EXP2.3.

5.2 EXP1.2 and EXP2.2: Effects of space optimization

As previously discussed, the limiting factor for these experiments is often not time complexity but rather space complexity. This limit is encountered while generating the input, before the algorithms themselves even start. The implementations discussed so far naively use the adjacency matrix in the form of a two-dimensional `double` array[12]. When running on a machine with 64 GB RAM, the optimal version of EXP1.1 and EXP2.1 encounters the upper space limit at input sizes of roughly 31,000 arguments, whereas the space-optimized version can handle up to approximately 34,000 arguments. However, the improvement is relatively small. Furthermore, this upper threshold is influenced by several factors, such as the density of the AAF graph (i. e. the cardinality of the attack relation) and of course by the overall available and allotted RAM size.

[11] Preliminary tests showed that this break-even point holds both on the MacBook with 8 CPU cores and FernUniversität server with only 4 CPU cores, as well as the FernUniversität server with 16 CPU cores which was used for the main tests.

[12] The data type `double` was used instead of `int` or other types for two reasons. On the one hand, the EJML `SimpleMatrix` requires a `double` or `float` input array. On the other hand, the data type `double` seemed better suited to handle overflow which can and did occur when the iterated matrix-matrix or matrix-vector products become too large (see also Section 5.4).

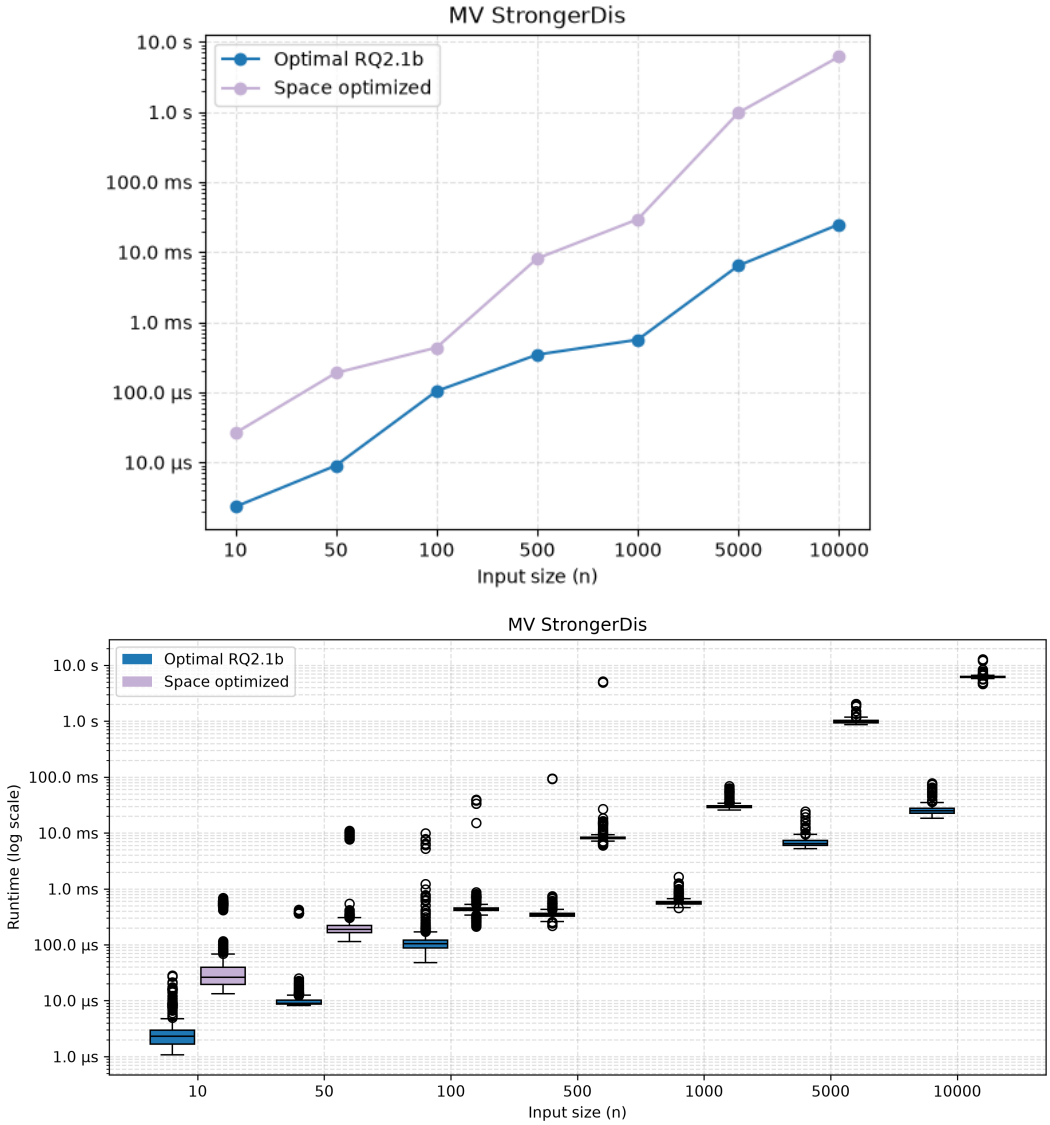

Figure 12: Comparison of the optimal version of EXP2.1 and the space-optimized version of the MV algorithm for solving StrongerDis.

In contrast, the slow-down is significant, as depicted in Figure 12. In most cases, the slow-down as compared to the optimal versions of EXP1.1 and EXP2.1 is more than one order of magnitude. This results in a performance which is sometimes even slower than the MM algorithms. For this reason and due to the difficulty to establish a general threshold for different hardware setups or even input densities, the space-

optimized implementation was not included for EXP1.3 and EXP2.3. Hence, the optimal version of the MV algorithms from the previous section will also be used for the next experiment.

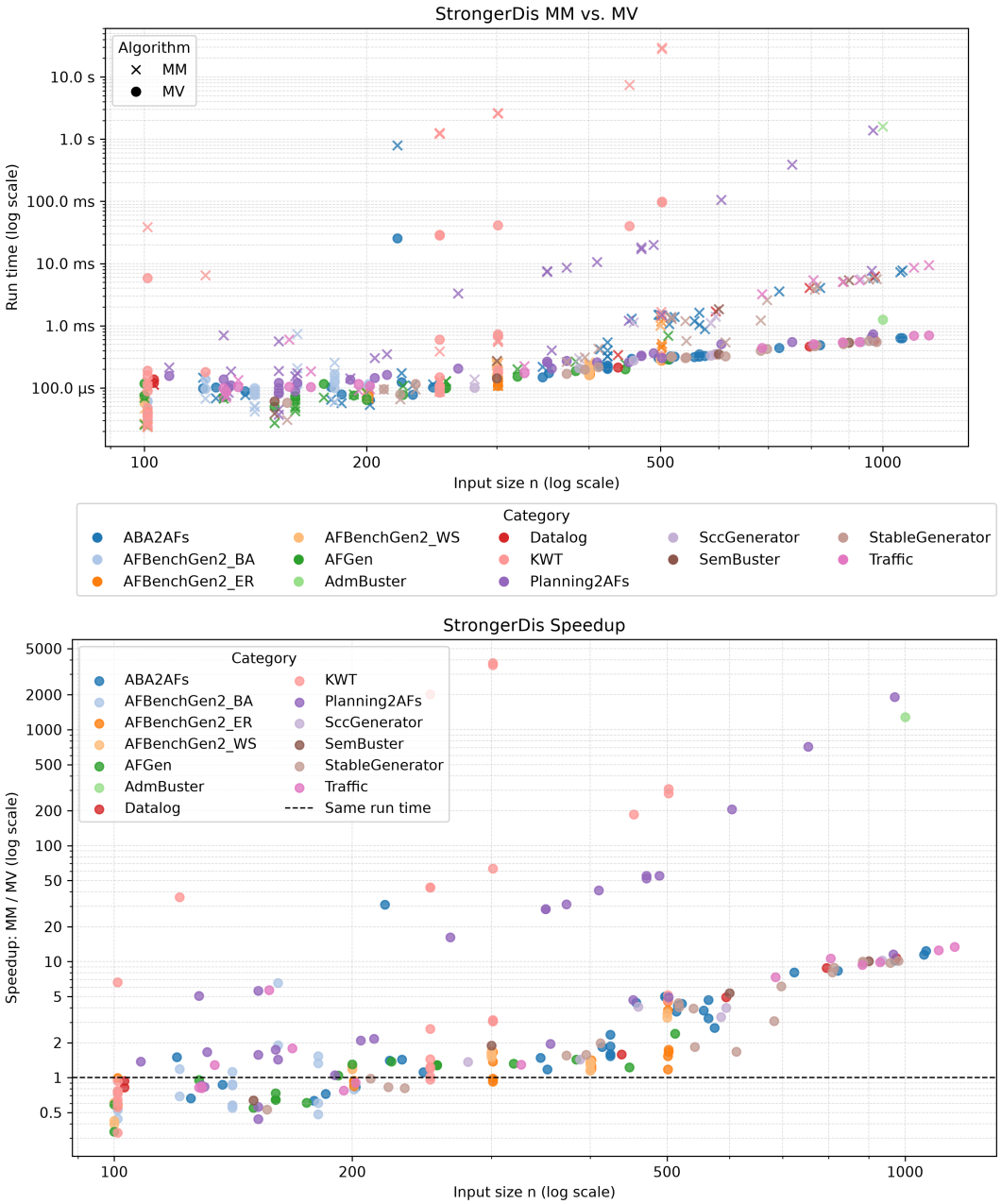

Figure 13: Comparison of the optimal MM and the MV implementations for solving StrongerDis.

5.3 EXP1.3 and 2.3: MM vs. MV

Figure 13 depicts the results for EXP2.3. The upper graph shows the median run time per ICCMA instance, while the lower graph shows the relative speed-up of the MV algorithm as compared to the MM algorithm. The graphs suggest that, with the exception of some outliers of the categories ABA2AFs and KWT, the run time is more heavily influenced by the input size than by the graph category. The speed-up of the MV algorithms also seem to depend strongly on the input size. While starting at approximately the input size of 200, the MV algorithm consistently outperforms the MM equivalent. In the range of input sizes of 100-200, there seems to be no clear tendency. The exception is the smallest input size of roughly 100 arguments, in which case the MM algorithms is faster in almost all instances. Remember that EXP1.1 and EXP2.1 showed that the input size 100 is the threshold at which parallelization outperforms the sequential implementation of the MV algorithm. It appears that in the comparison of the MM and the MV algorithms, the input size 100 also presents a relevant threshold. Furthermore, it is noteworthy that the speed-up remains generally within the range of up to a factor of 20. However, in several cases the speed-up rises to up to a factor of 50 or even almost 5000.

Both the MM and the MV algorithm present a polynomial time complexity. The upper graph overall generally forms the parabolic curves expected from this time complexity class. Especially the MV algorithm follows a slow-rising parabolic curve with the only outliers being certain KWT instances. The behavior of the MM algorithm, especially in input sizes below 200, appears to be less uniform, however, three parabolic trends can be observed. Interestingly, the two fast-rising trends correspond to specific categories; the upper salmon-colored trend corresponds to the KWT category, the dark purple middle trend is mostly comprised of Planning2AFs instances. The slowest-rising curve is formed by the remainder of the categories. This hints at the fact that the graph structure produced by the KWT and the Planning2AFs generators contains one or more features which are inherently more complex with regard to the computation of EQUIVDIS and STRONGERDIS. The discovery and study of these features might be an interesting topic of future research.

The graph properties of the instances producing the salmon-colored upper trend line, and the dark purple middle trend line are depicted in Tables 4 and 5. For the KWT trend line, one similarity among the instances is the high density between 0.95 and 1. While there are libraries that use specific methods for sparse matrix operations, in order to increase performance, crucially none of the implementations developed in this thesis make explicit use of sparse matrix operations. Therefore, this cannot explain why the remaining lower density instances (0,29 on average over all categories) perform better than the high density instances. On the other hand, the density might cause higher similarities between different arguments in the AAF, as the adjacency matrix is almost fully populated by 1. However, this is purely speculative, and requires further research. In addition, it can be observed than the maximum width and depth of the condensation graph is 2 or lower in all cases.

File	Nodes	Edges	Density	# SCCs	Largest SCC	Max width	Max depth
mainkwt_100_60_30_40_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	41	1561	0.9518	2	40	2	0
afinput_exp_cycles_indvary1_step1_batch_yyy04_bfs_3_sub05.af	220	47906	0.9943	2	219	1	1
mainkwt_250_100_50_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__1.af	151	22351	0.9868	2	150	2	0
mainkwt_250_150_75_100_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	101	9901	0.9803	2	100	2	0
mainkwt_250_150_75_100_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	101	9901	0.9803	2	100	2	0
mainkwt_250_150_75_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	151	22351	0.9868	2	150	2	0
mainkwt_250_150_75_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	151	22350	0.9868	2	150	2	0
mainkwt_250_150_75_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	151	22351	0.9868	2	150	2	0
mainkwt_500_200_100_300_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	301	89701	0.9934	2	300	2	0
mainkwt_500_300_150_300_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	301	89701	0.9934	2	300	2	0

Table 4: Instances of the upper trend line of the MM algorithm (salmon-colored dots and blue dot in Figure 13).

File	Nodes	Edges	Density	# SCCs	Largest SCC	Max width	Max depth
bw2.pfile-3-06.pddl.6.cnf.af	970	1768	0.0019	729	6	54	42
bw2.pfile-3-08.pddl.1.cnf.af	266	495	0.0070	224	2	21	30
ferry2.pfile-L2-C3-03.pddl.4.cnf.af	410	680	0.0041	329	2	26	31
ferry2.pfile-L2-C4-04.pddl.1.cnf.af	471	916	0.0041	402	2	62	30
ferry2.pfile-L2-C4-04.pddl.3.cnf.af	754	1336	0.0024	548	66	38	46
ferry2.pfile-L2-C4-06.pddl.1.cnf.af	471	916	0.0041	402	2	62	30
ferry2.pfile-L3-C1-010.pddl.3.cnf.af	373	647	0.0047	274	28	34	33
ferry2.pfile-L3-C1-06.pddl.2.cnf.af	351	663	0.0054	252	23	37	30
ferry2.pfile-L3-C1-07.pddl.2.cnf.af	351	663	0.0054	260	23	37	30
ferry2.pfile-L3-C3-05.pddl.1.cnf.af	489	949	0.0040	420	2	43	38
ferry2.pfile-L3-C4-09.pddl.2.cnf_bfs_1_sub05.af	604	1152	0.0032	544	21	87	37
admbuster_1000.af	1000	1498	0.0015	1000	1	2	500

Table 5: Instances of the middle trend line of the MM algorithm (dark purple dots and light green dot in Figure 13).

Instance	Run time
covington_2016-01.gml.20.af	447.8 ms
afinput_exp_cycles_indvary2_step1_batch_yyy06.af	432.6 ms
Medium-result_b26.af	424.7 ms
ferry2.pfile-L3-C4-09.pddl.2.cnf_community_3_sub08.af	355.5 ms
bw2.pfile-3-06.pddl.6.cnf.af	353.4 ms
empresa-publica-de-transportes-e-circulao_20141104_0243.gml.20.af	330.3 ms
durham-area-transit-authority_20150227_1734.gml.20.af	296.1 ms
afinput_exp_acyclic_indvary1_step2_batch_yyy07_random_1_sub08.af	264.5 ms
Medium-result_b28.af	249.2 ms
ferry2.pfile-L2-C4-04.pddl.3.cnf.af	215.5 ms

Table 6: Slowest run time of the 10 slowest instances for MV StrongerDis.

For the Planning2AFs dark purple trend line, on the other hand, it can be observed that the density is extremely low (< 0.01). While the possible similarity between arguments due to the sparsity of the matrix might be a contributing factor, this alone cannot hope to explain the higher run times, as there are several other instances with extremely low densities whose run times are part of the main trend line. The topic of the Planning2AFs outliers is therefore even more mysterious, and requires further analysis.

Furthermore, it can be observed that the MV algorithm appears to be much less affected by the input category. With the exception of a few KWT and one ABA2AFs instance, there are few outliers from the main trend line[13]. The MM algorithm, on the other hand, produces much more variation.

While Figure 13 shows that the median run time of most instances remains well under 1 second as in the previous experiments, the maximum run time of the 10 slowest instances is shown in Tables 6 and 7. The contrast is extreme; even the worst-case run times of the MV algorithm remain below 0.5 seconds. The worst-case run times of the MM algorithm require roughly 1.5 hours. This results in a difference roughly by factor 10,800. Furthermore, these results seem to confirm that the more iterations are required to obtain a result (i.e., the higher the overall run time), the larger becomes the difference between the MM and MV algorithms. Additionally, as the worst-case run-times for the MM and MV algorithms do not stem from the same instances in the same order (despite there being some overlap), it appears that these high run times are not uniquely attributable to the instance and its structure, but also seem to be influenced by the algorithm or implementation itself.

[13] A preliminary test using only the MV algorithm suggests that this trend line continues evenly for larger input sizes, as depicted in Appendix D.

Instance	Run time
afinput_exp_cycles_indvary2_step1_batch_yyy06.af	1.6 h
Medium-result_b26.af	1.5 h
covington_2016-01.gml.20.af	1.4 h
bw2.pfile-3-06.pddl.6.cnf.af	48.4 min
ferry2.pfile-L3-C4-09.pddl.2.cnf_community_3_sub08.af	43.8 min
empresa-publica-de-transportes-e-circulao_20141104_0243.gml.20.af	36.7 min
durham-area-transit-authority_20150227_1734.gml.20.af	27.0 min
afinput_exp_acyclic_indvary1_step2_batch_yyy07_random_1_sub08.af	18.1 min
Medium-result_b28.af	14.8 min
admbuster_1000.af	14.6 min

Table 7: Slowest run time of the 10 slowest instances for MM STRONGERDIS.

5.4 Limits to validity

This section discusses the limitations of the results presented in the previous sections. Some aspects concern the optimality of the implementations, whereas others directly influence the validity of the results, while at the same time yielding ideas for further work.

The first aspect regards the specific Java implementations used for the experiments. While there are certainly other possibilities to implement parallelization than the `parallel()` method of the `IntStream` class, as well as to implement sequential matrix operations than EJML, a comparison of all possible ways the algorithms could have been implemented would have gone beyond the scope of this thesis. The goal was to compare one sequential and one parallel option as a prototypical representative of their class. However, the selection of the EJML library and the `parallel()` method did undoubtedly affect the results in some way. More importantly, however, are the implementations within each class. A prominent example is the use of the `double` data type for the adjacency matrix and the column vectors. The initial idea was to use the `double` data type to avoid errors due to overflow of integer data types and therefore be able to accommodate large numbers produced by iterated matrix products. Additionally, the EJML `SimpleMatrix` constructor requires an input array of the `double` or `float` data type. However, this decision had two negative consequences.

First, while the idea to handle overflow might have been correct, the implementation was too generalized. The MM algorithms compute M^i for $i \leq 2|A| - 1$ and the resulting matrices therefore risk integer overflow, whereas the MV algorithms merely use the input adjacency matrix in its M^1 variant to iteratively multiply it with a column vector. Even though the sequential EJML implementation required at least the use of a `float` input, especially the parallelized MV implemenation could have employed, and might have benefited from the use of the `byte` or another space-efficient data type. Second, both the MM and the MV algorithms may produce

very large numbers in the adjacency matrix or column vector, respectively. In most cases, the `double` data type handled these large numbers effectively. However, one instance of the result validation for my Bachelorseminar [20] (which compares the MM and MV STRONGERDIS implementations in Java with implementations in Julia, Rust and C++) lead to the discovery that the adapted equality check explicitly implemented to catch possible rounding errors `if (Math.abs(sumV1 - sumV2) > 1e-9)` was not sufficient to avoid an incorrect return value. Unfortunately, this effect was only discovered after most of the experiments had already concluded with no time to repeat them. In future research, a more thorough consideration and treatment of integer overflow will be necessary.

The second aspect regards the worst-case performance of the MM algorithms. The theoretical run time complexity of the MV algorithm is $O(n^3)$, whereas the MM algorithm's is $O(n^4)$. However, this difference does not translate to the median run time, as seen in the previous sections. It does, however, heavily influence the run time of worst-case outliers and therefore the overall time required for the experiments. While the MV experiments terminate within less than one hour in most cases, the MM experiments ran for more than one day, sometimes several weeks without terminating. This suggests that the run time is influenced by the result of the algorithms, or rather the iteration in which the result is found. In the case of EQUIVDIS, a false result can be determined as early as in the first iteration. However, to determine a positive result, all $2|A| - 1$ iterations must be executed in order to establish this result. In the case of STRONGERDIS, the reverse applies; determining that one argument is stronger or weaker than another can happen from the first iteration on, whereas if the two arguments are equally strong, all iterations must be carried out. When informally reviewing the results of the algorithms for small input sizes, it appears that the equivalence of two arguments in randomly generated AAFs with the attack probability of 0.5 happens in less than 5% of instances. This fact might explain why in most cases both algorithms terminate relatively quickly (under 1 second), whereas only a small number of outliers exist which in turn illustrate the worst-case run time of several days.

The practical effect of this was that, in several cases, the experiment was aborted when a certain time was exceeded. The first experiment was started at the beginning of July, running only on 4 CPU cores[14]. This experiment was executing tests for EXP1.1 and ran for over 3 weeks without terminating. The experiment was then aborted and started anew with the newly allotted 16 CPU cores. In this case, the experiments for EXP1.1 terminated successfully within 2 days. However, over the course of the other experiments it occurred two more times that one single test instance ran for more than 48 hours without terminating. Unfortunately, the due date for this thesis was approaching, and again the choice was made to abort and retry. It is therefore crucial to note that the worst-case run times of several hours are not the absolute worst-case run times possible for these algorithms and implementa-

[14]This was due to the fact that access to the FernUniversität server was granted only at the end of June and initially only 4 CPU cores were alloted.

tions, but only the worst-case run times obtainable within the limited time frame of a bachelor's thesis (3 months). While the median run time is relatively robust to differences in the absolute run-time of a small number of worst-case test runs, this undoubtedly distorts the results of worst-case performances. This should be taken into account when interpreting the data.

Another aspect concerns the iteration threshold of the MV algorithms. As discussed in Section 3, the MV algorithms follow the same principle as the MM algorithms, and therefore the threshold should be $2|A| - 1$, as shown by Blümel et al. The implementation used in the experiments, however, was based on the algorithm by Kiefer et al. and consequently used a threshold of $2|A|$. Thus, the MV algorithms may perform one additional iteration in cases where the threshold is reached. This only affects instances for which the arguments are equivalent at the preceding iteration. Since such cases are rare as previously discussed, the impact on the measured run times is likely to be limited. Nevertheless, the results should be interpreted with this implementation detail in mind.

Finally, the fact that the algorithms' results (true or false) per test run and number of iterations after which it terminated were not recorded hindered possible insight in the effect of these parameters. While we might still guess at their importance, taking them into consideration when analyzing the test results would have certainly been advantageous.

6 Conclusion

This thesis investigated the computational complexity and practical run time of the discussion-based semantics for abstract argumentation. The work pursued two complementary goals: first, to determine whether the discussion-based semantics problems EQUIVDIS and STRONGERDIS can be solved within the NC complexity class, and second, to evaluate empirically how the resulting algorithms compare to the existing MM algorithms and how effectively their computations can be parallelized.

The theoretical results establish that EQUIVDIS is in NC. Building on the polynomial reduction chain established by Blümel et al., it was shown that both reduction steps from EQUIVDIS to EQUALWALKCOUNT to $\mathbb{Q}$-EQUIVALENCE can be computed in log space. Since $\mathbb{Q}$-EQUIVALENCE is in NC, this proves the NC membership of EQUIVDIS. Based on Kiefer et al.'s zeroness algorithm for $\mathbb{Q}$-automata, an optimized NC algorithm was then derived. Exploiting the highly restricted structure of the automata obtained from EQUIVDIS allowed several further simplifications and optimizations. In particular, the general $\mathbb{Q}$-automaton representation can be replaced by the adjacency matrix of the original AAF, and the randomized scalar introduced by the general zeroness algorithm can be omitted. This results in an algorithm based directly on iterated matrix-vector multiplications.

The same matrix-vector approach was subsequently used to develop an NC algorithm for STRONGERDIS. The proof of correctness demonstrates that the iterated multiplication of the adjacency matrix with the a-th basis vector computes the corresponding column a of successive powers of the adjacency matrix. The column sums therefore directly represent the numbers of walks relevant to the discussion counts. This establishes a close connection between the existing MM approach of Blümel et al. and the MV algorithms developed in this thesis: both ultimately operate on the same walk-based representation of discussion-based semantics, but the MV algorithms compute only the columns required to compare two selected arguments, whereas the MM approach computes the entire matrix required to establish the complete ranking.

The empirical evaluation demonstrates that theoretical parallelizability does translate into practical performance improvements, but that these improvements depend strongly on the input size. For the MV algorithms, parallelization is disadvantageous for small AAFs because the overhead of thread management and fork/join operations outweighs the computational benefit. The experiments indicate a break-even point at approximately 100 arguments. Above this threshold, the parallel implementations become increasingly beneficial, with improvements of approximately one order of magnitude on the median run time, observed from input sizes of around 500 arguments onwards. In contrast, the MM algorithms benefit from parallelization already at comparatively small input sizes of 10 arguments, presumably because their more expensive matrix-matrix multiplications provide a sufficiently large computational workload to compensate for the parallelization overhead.

The space-optimized implementation produced a less favorable result. Although avoiding an explicitly stored adjacency matrix increased the maximum feasible input size on the test environment from approximately 31,000 to 34,000 arguments, the improvement was comparatively small. At the same time, the resulting run time was in most cases more than one order of magnitude higher than that of the optimal implementation. Consequently, the space-optimized implementation did not provide a sufficiently attractive trade-off between memory consumption and computational performance to be used for the final comparison. These results also emphasize that the limiting resource for the investigated implementations is often memory rather than computation time.

Another important observation regards the difference between typical and worst-case behavior. Although the median run times of the tested implementations were generally below 1 second, individual instances could require up to several hours or days. This behavior is consistent with the structure of the two decision problems. In EQUIVDIS, a negative result can be established as soon as a difference in discussion counts is found, whereas a positive result requires all relevant iterations to be examined. For STRONGERDIS, the situation is reversed for arguments that are equivalent. The resulting small number of difficult instances can therefore have a substantial effect on total computation time, even while the median run time remains low. The experiments further show that the difference in time complexity between the $O(n^3)$ MV algorithm and the $O(n^4)$ MM algorithm is particularly relevant for these instances.

The results of the experiments on the ICCMA 2025 instances suggest that while in most cases, the input size is the most relevant factor determining the median run time, the graph structure of the input appears to influence the results in certain cases. Especially the categories KWT and Planning2AFs seem to contain structural properties which significantly slow down the computation of EQUIVDIS and STRONGERDIS.

The results should nevertheless be interpreted in light of the limitations discussed in this thesis. In particular, the empirical comparison concerns specific Java implementations rather than the abstract algorithms independently of their implementation. The use of EJML, Java parallel streams, fixed attack probabilities, and the `double` data type can all influence the measured performance. The latter is particularly relevant for large instances, where the values resulting from repeated matrix operations can become very large and rounding errors may affect the correctness of comparisons. Furthermore, the time constraints of the thesis required several extremely long-running experiments to be aborted. The reported worst-case behavior should therefore not be interpreted as an absolute upper bound on the run time of the algorithms.

Despite these limitations, the thesis provides both a theoretical and an empirical contribution to the study of discussion-based semantics. The theoretical results establish NC time complexity and provide concrete algorithms whose structure is particularly well suited to parallel computation. The empirical results, in turn, demon-

strate that this theoretical property can translate into significant practical benefits for sufficiently large instances, while also illustrating the importance of parallelization overhead, memory consumption, and implementation details. More generally, the results highlight that theoretical complexity alone is not enough to predict the practical performance of algorithms for ranking-based semantics.

Several directions for future research follow naturally from these findings. A first step would be to investigate alternative representations of the attack relation and more memory-efficient data types, particularly for the MV algorithms. A second direction would be a systematic comparison of different parallelization strategies and implementations, including approaches beyond Java's parallel stream framework. Finally, a more comprehensive empirical study, run on more powerful hardware and using a larger selection of ICCMA instances as well as a more systematic analysis of graph properties such as density, cyclicity, and strongly connected components could help explain the substantial variation in run time observed between individual instances. Such an investigation could provide a clearer picture of when the theoretical advantages of MV algorithms translate into practical advantages for discussion-based semantics.

In conclusion, the results of this thesis demonstrate that discussion-based semantics are not only decidable in polynomial time but can also be approached from the perspective of efficient parallel computation. The developed MV algorithms provide a more targeted alternative to the existing approach by computing only the information required for the comparison of selected arguments. At the same time, the empirical evaluation shows that the practical value of this approach depends on the characteristics of the input and the implementation.

References

[1] Leila Amgoud and Jonathan Ben-Naim. Ranking-Based Semantics for Argumentation Frameworks. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, Weiru Liu, V. S. Subrahmanian, and Jef Wijsen, editors, *Scalable Uncertainty Management*, volume 8078, pages 134–147. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013. Series Title: Lecture Notes in Computer Science. doi:10.1007/978-3-642-40381-1_11.

[2] Iosif Apostolakis, Andrei Popescu, and Johannes P. Wallner. *Solver and Benchmark Descriptions of ICCMA 2025*. Proceedings of the ICCMA competition. 2025. URL: https://www.argumentationcompetition.org/2025/iccma-25-proceedings.pdf.

[3] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, Cambridge, 2009. OCLC: 443221176.

[4] T.J.M. Bench-Capon and Paul E. Dunne. Argumentation in artificial intelligence. *Artificial Intelligence*, 171(10-15):619–641, July 2007. doi:10.1016/j.artint.2007.05.001.

[5] Lydia Blümel, Kai Sauerwald, Kenneth Skiba, and Matthias Thimm. On the Complexity of the Discussion-based Semantics in Abstraction Argumentation, 2026. Version Number: 1. doi:10.48550/ARXIV.2604.11480.

[6] Elise Bonzon, Jérôme Delobelle, Sébastien Konieczny, and Nicolas Maudet. An empirical and axiomatic comparison of ranking-based semantics for abstract argumentation. *Journal of Applied Non-Classical Logics*, 33(3-4):328–386, October 2023. doi:10.1080/11663081.2023.2246863.

[7] Diego Costa, Cor-Paul Bezemer, Philipp Leitner, and Artur Andrzejak. What's Wrong with My Benchmark Results? Studying Bad Practices in JMH Benchmarks. *IEEE Transactions on Software Engineering*, 47(7):1452–1467, July 2021. doi:10.1109/TSE.2019.2925345.

[8] Jérôme Delobelle. *Ranking-based Semantics for Abstract Argumentation*. PhD thesis, Centre de Recherche en Informatique de Lens - Université d'Artois, 2017.

[9] Jérôme Delobelle. Personal communication, April 2026.

[10] Pierpaolo Dondio. Ranking Semantics Based on Subgraphs Analysis. 2018. doi:10.21427/9HCS-1S56.

[11] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, September 1995. doi:10.1016/0004-3702(94)00041-X.

[12] Sarah A. Gaggl, Thomas Linsbichler, Marco Maratea, and Stefan Woltran. Design and results of the Second International Competition on Computational Models of Argumentation. *Artificial Intelligence*, 279:103193, February 2020. URL: https://linkinghub.elsevier.com/retrieve/pii/S0004370218302029, doi:10.1016/j.artint.2019.103193.

[13] William Hesse, Eric Allender, and David A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences*, 65(4):695–716, December 2002. doi:10.1016/S0022-0000(02)00025-9.

[14] Marcell Jawhari. *An empirical correlation analysis of ranking semantics for abstract argumentation frameworks*. Bachelor's thesis, FernUniversität in Hagen, 2025.

[15] Aleksandr Kartuzov, Tatyana Kartuzova, and Marina Sirotkina. Parallel Programming Methods for High-Performance Tasks in Java. In *2026 International Russian Smart Industry Conference (SmartIndustryCon)*, pages 239–244, Sochi, Russian Federation, March 2026. IEEE. doi:10.1109/SmartIndustryCon68821.2026.11492905.

[16] Stefan Kiefer. Notes on Equivalence and Minimization of Weighted Automata, 2020. Version Number: 1. doi:10.48550/ARXIV.2009.01217.

[17] Stefan Kiefer, Andrzej Murawski, Joel Ouaknine, Bjoern Wachter, and James Worrell. On the Complexity of Equivalence and Minimisation for Q-weighted Automata. *Logical Methods in Computer Science*, Volume 9, Issue 1:908, March 2013. doi:10.2168/LMCS-9(1:8)2013.

[18] Stefan Kiefer, Andrzej S. Murawski, Joël Ouaknine, Björn Wachter, and James Worrell. On the Complexity of the Equivalence Problem for Probabilistic Automata. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, and Lars Birkedal, editors, *Foundations of Software Science and Computational Structures*, volume 7213, pages 467–481. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. Series Title: Lecture Notes in Computer Science. doi:10.1007/978-3-642-28729-9_31.

[19] Robert A. Kowalski and Francesca Toni. Abstract argumentation. *Artificial Intelligence and Law*, 4(3-4):275–296, 1996. doi:10.1007/BF00118494.

[20] Vanessa Köbke. *Die Programmiersprache Julia*. Seminararbeit - Bachelorseminar Programmiersysteme, FernUniversität in Hagen, Hagen, August 2026. URL: https://github.com/vanessakoebke/BachelorThesis/main/documentation/Seminararbeit.pdf.

[21] Oracle. Interface IntStream, 2025. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/IntStream.html.

[22] Oracle. Package java.util.stream, 2025. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html#StreamOps.

[23] Stephan Ritscher. *Degree Bounds and Complexity of Gröbner Bases of Important Classes of Polynomial Ideals*. PhD thesis, Technische Universität München, München, December 2010. URL: https://mediatum.ub.tum.de/doc/1006213/1006213.pdf?utm_source=chatgpt.com.

[24] Eduardo Rosales, Matteo Basso, Andrea Rosà, and Walter Binder. Large-scale characterization of Java streams. *Software: Practice and Experience*, 53(9):1763–1792, September 2023. doi:10.1002/spe.3213.

[25] André Schulz. *Grundlagen der Theoretischen Informatik*. Springer Berlin Heidelberg, Berlin, Heidelberg, 1. aufl. 2022 edition, 2022. doi:10.1007/978-3-662-65142-1.

[26] J. T. Schwartz. Fast Probabilistic Algorithms for Verification of Polynomial Identities. *Journal of the ACM*, 27(4):701–717, October 1980. doi:10.1145/322217.322225.

[27] Michael Sipser. *Introduction to the theory of computation*. Cengage Learning, Australia Brazil Japan Korea Mexiko Singapore Spain United Kingdom United States, third edition, international edition edition, 2013.

[28] Antonio Trovato, Luca Traini, Federico Di Menna, and Dario Di Nucci. AMBER: An AI-enabled Java Microbenchmark Harness Extension to Dynamically Terminate Warm-up Iterations. *Science of Computer Programming*, 253:103487, August 2026. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642326000535, doi:10.1016/j.scico.2026.103487.

[29] Wen-Guey Tzeng. On path equivalence of nondeterministic finite automata. *Information Processing Letters*, 58(1):43–46, April 1996. doi:10.1016/0020-0190(96)00039-7.

[30] Douglas Walton. Argumentation Theory: A Very Short Introduction. In Guillermo Simari and Iyad Rahwan, editors, *Argumentation in Artificial Intelligence*, pages 1–22. Springer US, Boston, MA, 2009. doi:10.1007/978-0-387-98197-0_1.

[31] R. E. Zippel. *Probabilistic algorithms for sparse polynomials*. 1979. URL: https://searchworks.stanford.edu/view/4589819.

A Input Files

File	Category
afinput_exp_acyclic_depvary_step5_batch_yyy02.af	ABA2AFs
afinput_exp_acyclic_depvary_step6_batch_yyy01.af	ABA2AFs
afinput_exp_acyclic_depvary_step6_batch_yyy04.af	ABA2AFs
afinput_exp_acyclic_depvary_step6_batch_yyy08.af	ABA2AFs
afinput_exp_acyclic_depvary_step6_batch_yyy09.af	ABA2AFs
afinput_exp_acyclic_depvary_step8_batch_yyy09.af	ABA2AFs
afinput_exp_acyclic_indvary1_step1_batch_yyy03.af	ABA2AFs
afinput_exp_acyclic_indvary1_step1_batch_yyy05.af	ABA2AFs
afinput_exp_acyclic_indvary1_step2_batch_yyy07_community_3_sub05.af	ABA2AFs
afinput_exp_acyclic_indvary1_step2_batch_yyy07_random_1_sub08.af	ABA2AFs
afinput_exp_acyclic_indvary2_step1_batch_yyy03.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy02_random_4_sub05.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy09.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy10_bfs_4_sub05.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy10_community_1_sub05.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy10_community_3_sub05.af	ABA2AFs
afinput_exp_acyclic_indvary2_step2_batch_yyy10_community_4_sub05.af	ABA2AFs
afinput_exp_cycles_depvary_step3_batch_yyy08_bfs_1_sub08.af	ABA2AFs
afinput_exp_cycles_depvary_step3_batch_yyy09.af	ABA2AFs
afinput_exp_cycles_depvary_step4_batch_yyy02.af	ABA2AFs
afinput_exp_cycles_depvary_step4_batch_yyy03.af	ABA2AFs
afinput_exp_cycles_depvary_step4_batch_yyy07_bfs_5_sub05.af	ABA2AFs
afinput_exp_cycles_depvary_step4_batch_yyy07_community_2_sub05.af	ABA2AFs
afinput_exp_cycles_depvary_step6_batch_yyy02.af	ABA2AFs
afinput_exp_cycles_indvary1_step1_batch_yyy04_bfs_3_sub05.af	ABA2AFs
afinput_exp_cycles_indvary1_step1_batch_yyy04_community_2_sub08.af	ABA2AFs
afinput_exp_cycles_indvary1_step3_batch_yyy02.af	ABA2AFs
afinput_exp_cycles_indvary1_step4_batch_yyy10.af	ABA2AFs
afinput_exp_cycles_indvary2_step1_batch_yyy06.af	ABA2AFs
afinput_exp_cycles_indvary2_step1_batch_yyy10_random_2_sub05.af	ABA2AFs
afinput_exp_cycles_indvary3_step1_batch_yyy04.af	ABA2AFs
afinput_exp_cycles_indvary3_step1_batch_yyy09.af	ABA2AFs
BA_100_10_3.af	AFBenchGen2_BA
BA_100_40_5.af	AFBenchGen2_BA
BA_100_90_2.af	AFBenchGen2_BA
BA_120_30_3.af	AFBenchGen2_BA
BA_120_90_4.af	AFBenchGen2_BA
BA_140_10_3.af	AFBenchGen2_BA
BA_140_20_4.af	AFBenchGen2_BA
BA_140_40_5.af	AFBenchGen2_BA
BA_140_50_4.af	AFBenchGen2_BA
BA_140_60_1.af	AFBenchGen2_BA
BA_140_60_2.af	AFBenchGen2_BA
BA_160_80_2.af	AFBenchGen2_BA
BA_160_90_2.af	AFBenchGen2_BA

BA_180_40_3.af	AFBenchGen2_BA
BA_180_80_4.af	AFBenchGen2_BA
BA_180_80_5.af	AFBenchGen2_BA
BA_180_90_2.af	AFBenchGen2_BA
BA_200_0_3.af	AFBenchGen2_BA
BA_200_60_4.af	AFBenchGen2_BA
BA_200_70_3.af	AFBenchGen2_BA
ER_100_20_4.af	AFBenchGen2_ER
ER_100_90_9.af	AFBenchGen2_ER
ER_200_20_3.af	AFBenchGen2_ER
ER_200_20_5.af	AFBenchGen2_ER
ER_200_70_1.af	AFBenchGen2_ER
ER_300_20_2.af	AFBenchGen2_ER
ER_300_30_4.af	AFBenchGen2_ER
ER_300_40_3.af	AFBenchGen2_ER
ER_300_50_8.af	AFBenchGen2_ER
ER_300_60_4.af	AFBenchGen2_ER
ER_400_60_3.af	AFBenchGen2_ER
ER_400_70_7.af	AFBenchGen2_ER
ER_400_80_3.af	AFBenchGen2_ER
ER_500_100_4.af	AFBenchGen2_ER
ER_500_100_5.af	AFBenchGen2_ER
ER_500_10_10.af	AFBenchGen2_ER
ER_500_40_1.af	AFBenchGen2_ER
ER_500_50_1.af	AFBenchGen2_ER
ER_500_50_3.af	AFBenchGen2_ER
ER_500_90_2.af	AFBenchGen2_ER
WS_100_12_30_70.af	AFBenchGen2_WS
WS_100_12_70_50.af	AFBenchGen2_WS
WS_100_24_90_30.af	AFBenchGen2_WS
WS_200_14_70_30.af	AFBenchGen2_WS
WS_200_20_10_50.af	AFBenchGen2_WS
WS_200_20_30_70.af	AFBenchGen2_WS
WS_200_28_70_70.af	AFBenchGen2_WS
WS_300_16_50_10.af	AFBenchGen2_WS
WS_300_16_90_30.af	AFBenchGen2_WS
WS_300_32_30_70.af	AFBenchGen2_WS
WS_300_32_90_70.af	AFBenchGen2_WS
WS_300_8_10_30.af	AFBenchGen2_WS
WS_300_8_90_10.af	AFBenchGen2_WS
WS_400_16_10_10.af	AFBenchGen2_WS
WS_400_16_10_70.af	AFBenchGen2_WS
WS_400_16_90_50.af	AFBenchGen2_WS
WS_400_32_50_50.af	AFBenchGen2_WS
WS_400_8_90_30.af	AFBenchGen2_WS
WS_500_32_10_50.af	AFBenchGen2_WS
WS_500_32_50_10.af	AFBenchGen2_WS
n100p3q34ve.af	AFGen
n100p5q2.af	AFGen

n128p5q2_ve.af	AFGen
n150p3q34n.af	AFGen
n160p3q24e.af	AFGen
n160p3q34h.af	AFGen
n160p5q2_ve.af	AFGen
n175p3q34h.af	AFGen
n192p5q2_ve.af	AFGen
n200p3q34h.af	AFGen
n224p5q2_ve.af	AFGen
n256p3q08n.af	AFGen
n256p5q2_e.af	AFGen
n320p5q2_n.af	AFGen
n384p5q2_vh.af	AFGen
n448p5q2_vh.af	AFGen
n512p5q2_vh.af	AFGen
admbuster_1000.af	AdmBuster
Medium-result_b1.af	Datalog
Medium-result_b17.af	Datalog
Medium-result_b26.af	Datalog
Medium-result_b28.af	Datalog
Small-result_b89.af	Datalog
Small-result_b93.af	Datalog
mainkwt_100_40_20_10_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_100_40_20_10_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_100_40_20_10_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_100_40_20_40_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_100_40_20_60_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_100_60_30_10_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_100_60_30_40_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__1.af	KWT
mainkwt_100_60_30_40_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	KWT
mainkwt_100_60_30_40_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_100_60_30_60_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_250_100_50_100_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_250_100_50_100_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_250_100_50_100_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_250_100_50_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	KWT
mainkwt_250_100_50_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__1.af	KWT
mainkwt_250_100_50_25_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_250_150_75_100_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_250_150_75_100_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_250_150_75_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_250_150_75_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	KWT
mainkwt_250_150_75_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_250_150_75_150_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_250_150_75_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__3.af	KWT
mainkwt_250_150_75_150_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_250_150_75_25_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__2.af	KWT
mainkwt_500_200_100_200_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_500_200_100_300_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT

mainkwt_500_200_100_50_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__5.af	KWT
mainkwt_500_300_150_300_100_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
mainkwt_500_300_150_300_200_0.4_0.3_0.4_0.3_0.4_0.3_0.2__4.af	KWT
bw2.pfile-3-05.pddl.2.cnf_bfs_5_sub08.af	Planning2AFs
bw2.pfile-3-06.pddl.6.cnf.af	Planning2AFs
bw2.pfile-3-08.pddl.1.cnf.af	Planning2AFs
bw2.pfile-3-08.pddl.2.cnf.af	Planning2AFs
bw2.pfile-3-08.pddl.2.cnf_random_1_sub08.af	Planning2AFs
ferry2.pfile-L2-C1-02.pddl.2.cnf_bfs_1_sub08.af	Planning2AFs
ferry2.pfile-L2-C1-04.pddl.2.cnf.af	Planning2AFs
ferry2.pfile-L2-C1-07.pddl.2.cnf.af	Planning2AFs
ferry2.pfile-L2-C1-09.pddl.3.cnf.af	Planning2AFs
ferry2.pfile-L2-C1-09.pddl.5.cnf.af	Planning2AFs
ferry2.pfile-L2-C2-08.pddl.1.cnf_degree_1_sub08.af	Planning2AFs
ferry2.pfile-L2-C2-09.pddl.3.cnf.af	Planning2AFs
ferry2.pfile-L2-C3-010.pddl.1.cnf_bfs_2_sub08.af	Planning2AFs
ferry2.pfile-L2-C3-02.pddl.5.cnf.af	Planning2AFs
ferry2.pfile-L2-C3-03.pddl.4.cnf.af	Planning2AFs
ferry2.pfile-L2-C3-05.pddl.6.cnf.af	Planning2AFs
ferry2.pfile-L2-C3-07.pddl.1.cnf_community_3_sub08.af	Planning2AFs
ferry2.pfile-L2-C3-08.pddl.1.cnf.af	Planning2AFs
ferry2.pfile-L2-C3-08.pddl.1.cnf_degree_2_sub08.af	Planning2AFs
ferry2.pfile-L2-C3-08.pddl.1.cnf_random_4_sub08.af	Planning2AFs
ferry2.pfile-L2-C4-04.pddl.1.cnf.af	Planning2AFs
ferry2.pfile-L2-C4-04.pddl.3.cnf.af	Planning2AFs
ferry2.pfile-L2-C4-06.pddl.1.cnf.af	Planning2AFs
ferry2.pfile-L3-C1-010.pddl.3.cnf.af	Planning2AFs
ferry2.pfile-L3-C1-06.pddl.2.cnf.af	Planning2AFs
ferry2.pfile-L3-C1-07.pddl.2.cnf.af	Planning2AFs
ferry2.pfile-L3-C3-05.pddl.1.cnf.af	Planning2AFs
ferry2.pfile-L3-C4-09.pddl.2.cnf_bfs_1_sub05.af	Planning2AFs
ferry2.pfile-L3-C4-09.pddl.2.cnf_community_3_sub08.af	Planning2AFs
scc_280_15_0.4_0.1_8.af	SccGenerator
scc_388_5_0.3_0.1_13.af	SccGenerator
scc_460_19_0.3_0.05_4.af	SccGenerator
scc_585_8_0.3_0.05_9.af	SccGenerator
scc_594_44_0.4_0.2_1.af	SccGenerator
scc_935_33_0.3_0.05_14.af	SccGenerator
sembuster_150.af	SemBuster
sembuster_300.af	SemBuster
sembuster_600.af	SemBuster
sembuster_900.af	SemBuster
st_156_23_11_28_43.af	StableGenerator
st_211_11_10_7_31.af	StableGenerator
st_222_24_11_7_50.af	StableGenerator
st_233_8_29_17_28.af	StableGenerator
st_373_25_13_28_27.af	StableGenerator
st_395_23_21_18_90.af	StableGenerator
st_412_21_34_36_17.af	StableGenerator

st_517_7_26_27_38.af	StableGenerator
st_518_6_24_14_84.af	StableGenerator
st_540_25_12_12_9.af	StableGenerator
st_542_25_17_36_39.af	StableGenerator
st_612_12_22_39_36.af	StableGenerator
st_683_5_18_37_65.af	StableGenerator
st_697_18_35_34_1.af	StableGenerator
st_809_25_10_17_46.af	StableGenerator
st_813_22_36_6_88.af	StableGenerator
st_884_6_17_34_91.af	StableGenerator
st_885_12_24_37_68.af	StableGenerator
st_958_20_26_8_11.af	StableGenerator
st_981_11_33_9_18.af	StableGenerator
beaumont-pass-transit_20151216_1656.gml.50.af	Traffic
corona_ca_2015-12-21.gml.20.af	Traffic
covington_2016-01.gml.20.af	Traffic
des-moines-area-regional-transit-authority_20150715_1946.gml.50.af	Traffic
durham-area-transit-authority_20150227_1734.gml.20.af	Traffic
empresa-publica-de-transportes-e-circulao_20141104_0243.gml.20.af	Traffic
go-taps_20150524_1019.gml.20_community_3_sub08.af	Traffic
laketahoe-ca-us-archiver_20151216_1536.gml.80.af	Traffic
leon_spain_2016-01.gml.80.af	Traffic
lincolncounty_or_2015-12-03.gml.80.af	Traffic
lowell-regional-transit-authority_20121214_0433.gml.20.af	Traffic
mrc-les-moulins-urbis_20151202_1947.gml.50.af	Traffic
sonomacounty-ca-us-archiver_20140630_0109.gml.80.af	Traffic
thecomet_20131025_1906.gml.20_random_3_sub08.af	Traffic
yamhill-or-us.gml.80.af	Traffic

Table 8: Input files for EXP1.3 and EXP2.3

B Results of Experiment 1: EQUIVDIS

B.1 EXP 1.1: parallelization

B.1.1 EXP1.1a: MM algorithm

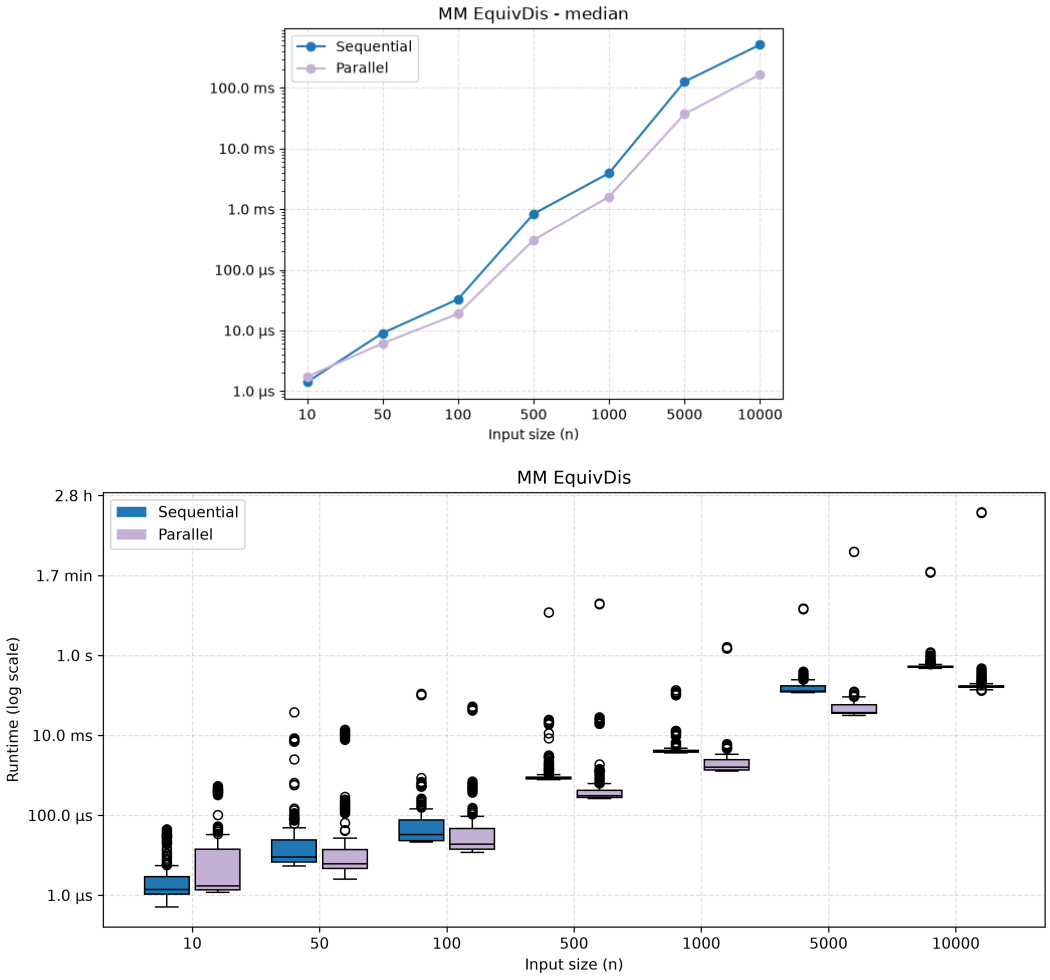

Figure 14: Comparison of the sequential and parallel implementations of the MM algorithm for solving EQUIVDIS.

B.1.2 EXP1.1b: MV algorithm

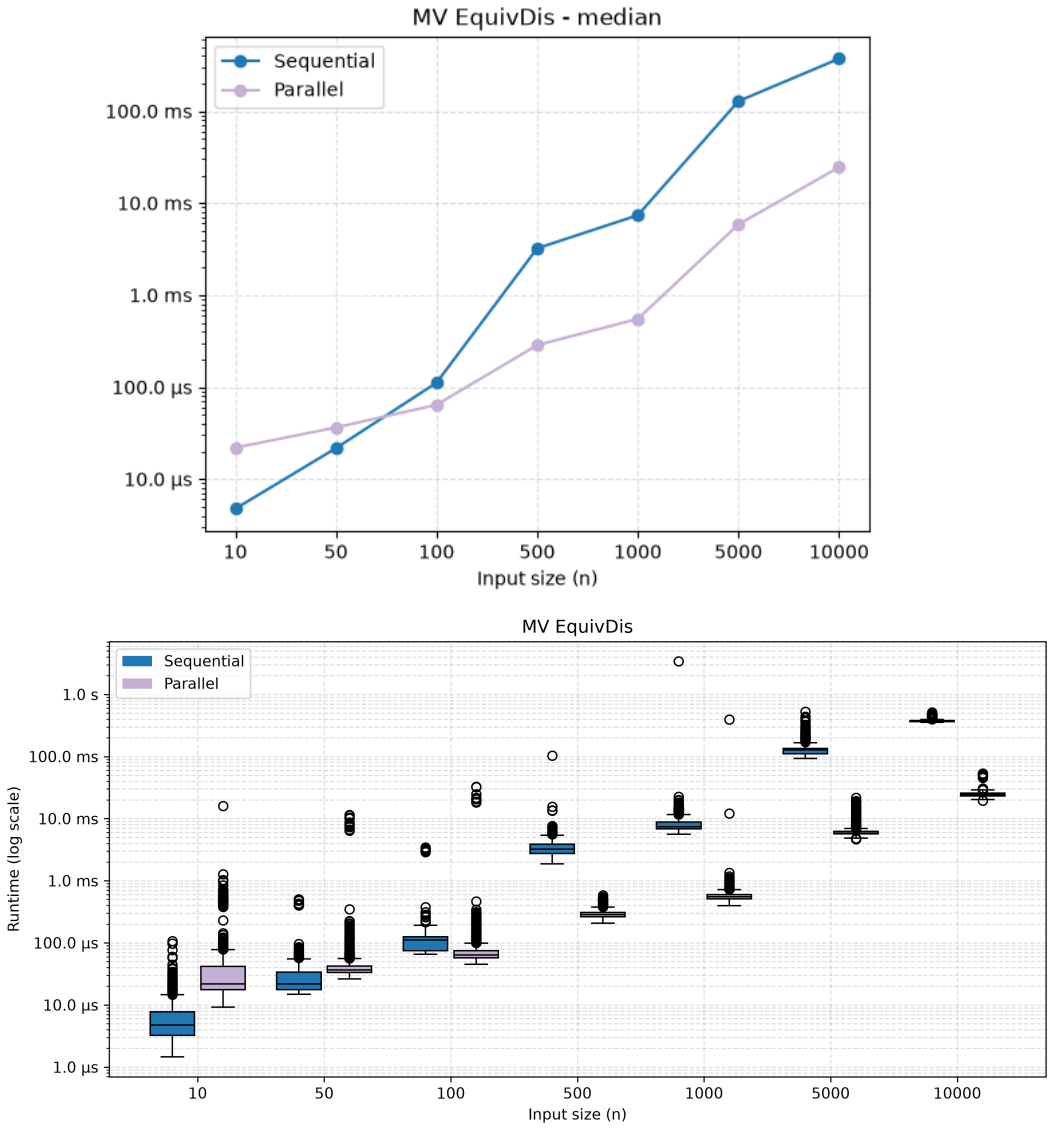

Figure 15: Comparison of the sequential and parallel implementations of the MV algorithm for solving EQUIVDIS.

B.2 EXP1.2: Space optimization

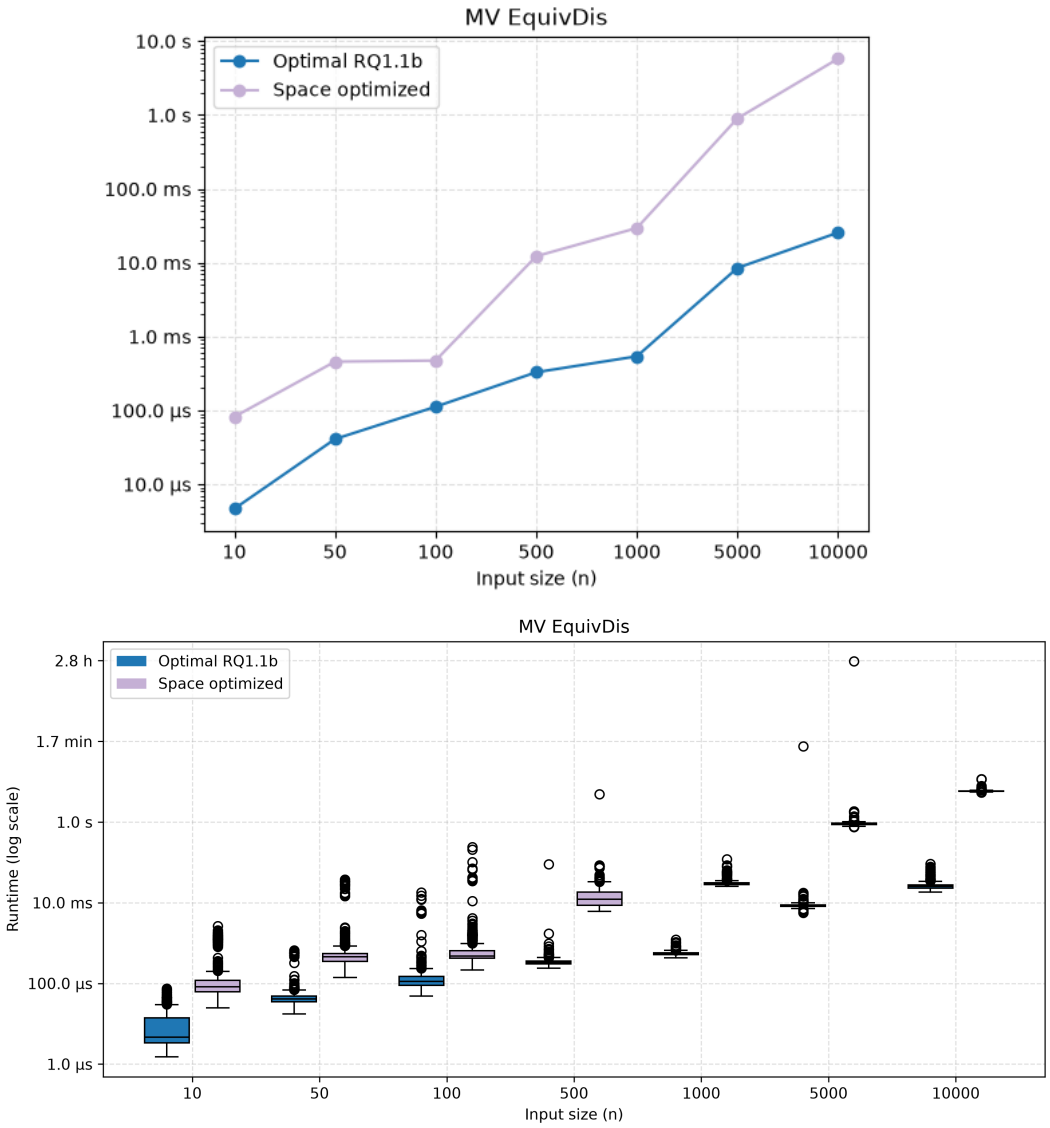

Figure 16: Comparison of the optimal version of EXP1.1 and the space-optimized version of the MM algorithm for solving EQUIVDIS.

B.3 EXP1.3: MM vs. MV

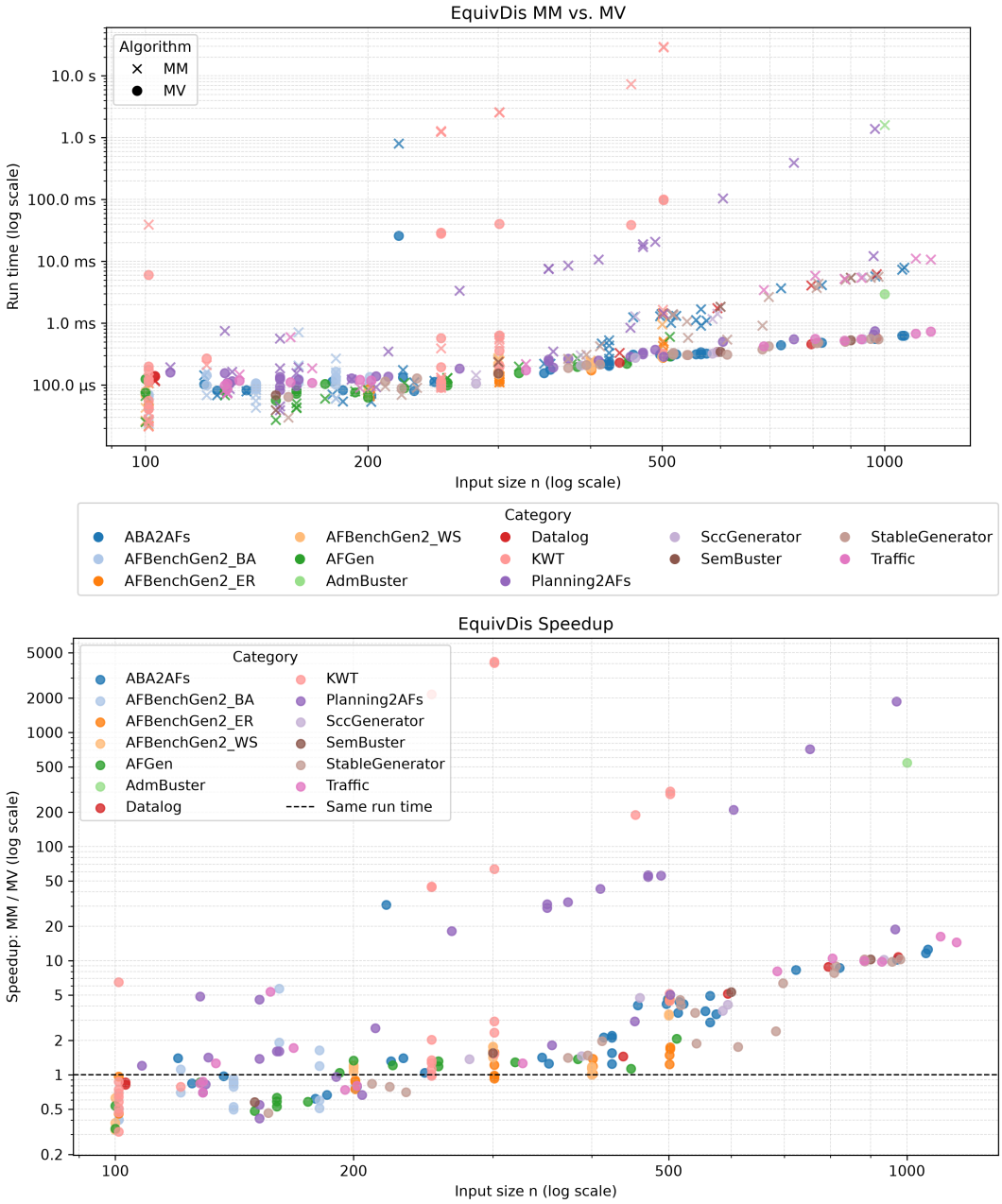

Figure 17: Comparison of the optimal MM and the MV implementations for solving EQUIVDIS.

C Results of Experiment 2: StrongerDis

C.1 EXP2.1: parallelization

C.1.1 EXP2.1a: MM algorithm

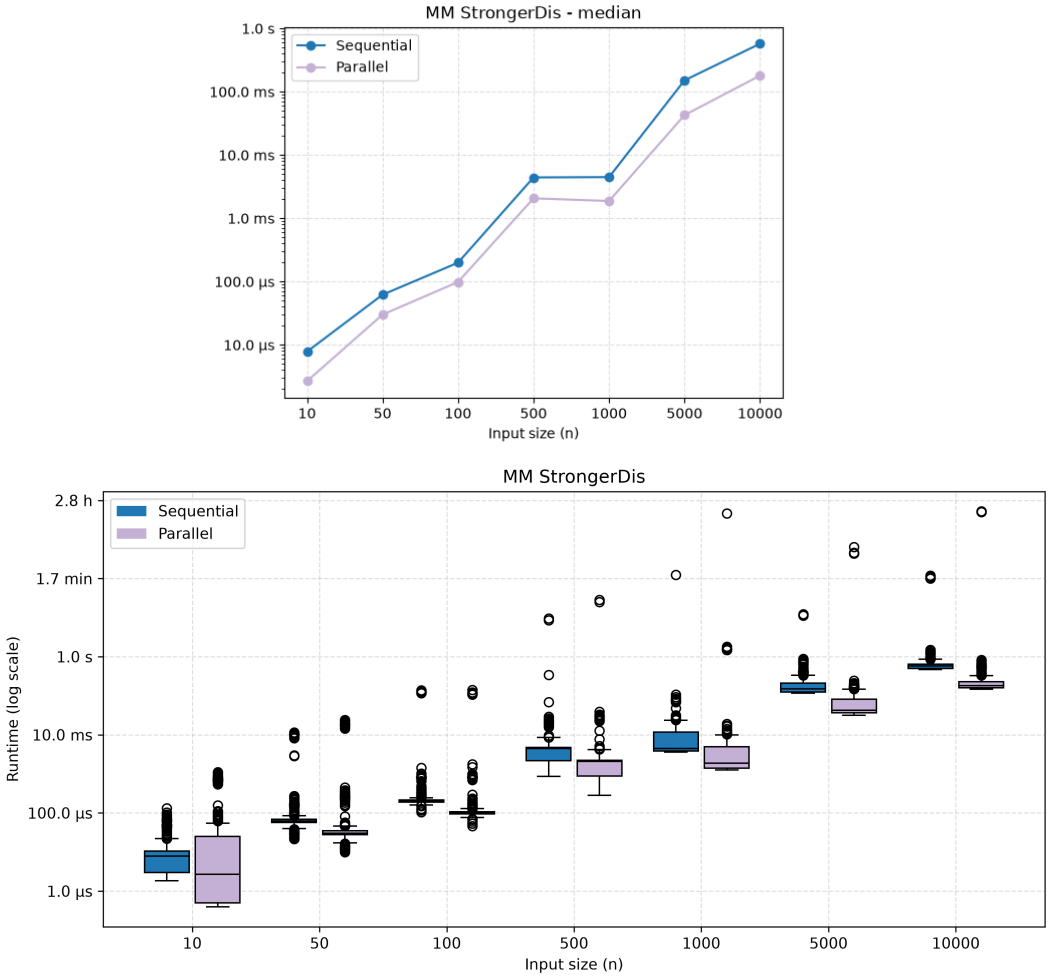

Figure 18: Comparison of the sequential and parallel implementations of the MM algorithm for solving StrongerDis.

C.1.2 EXP2.1b: MV algorithm

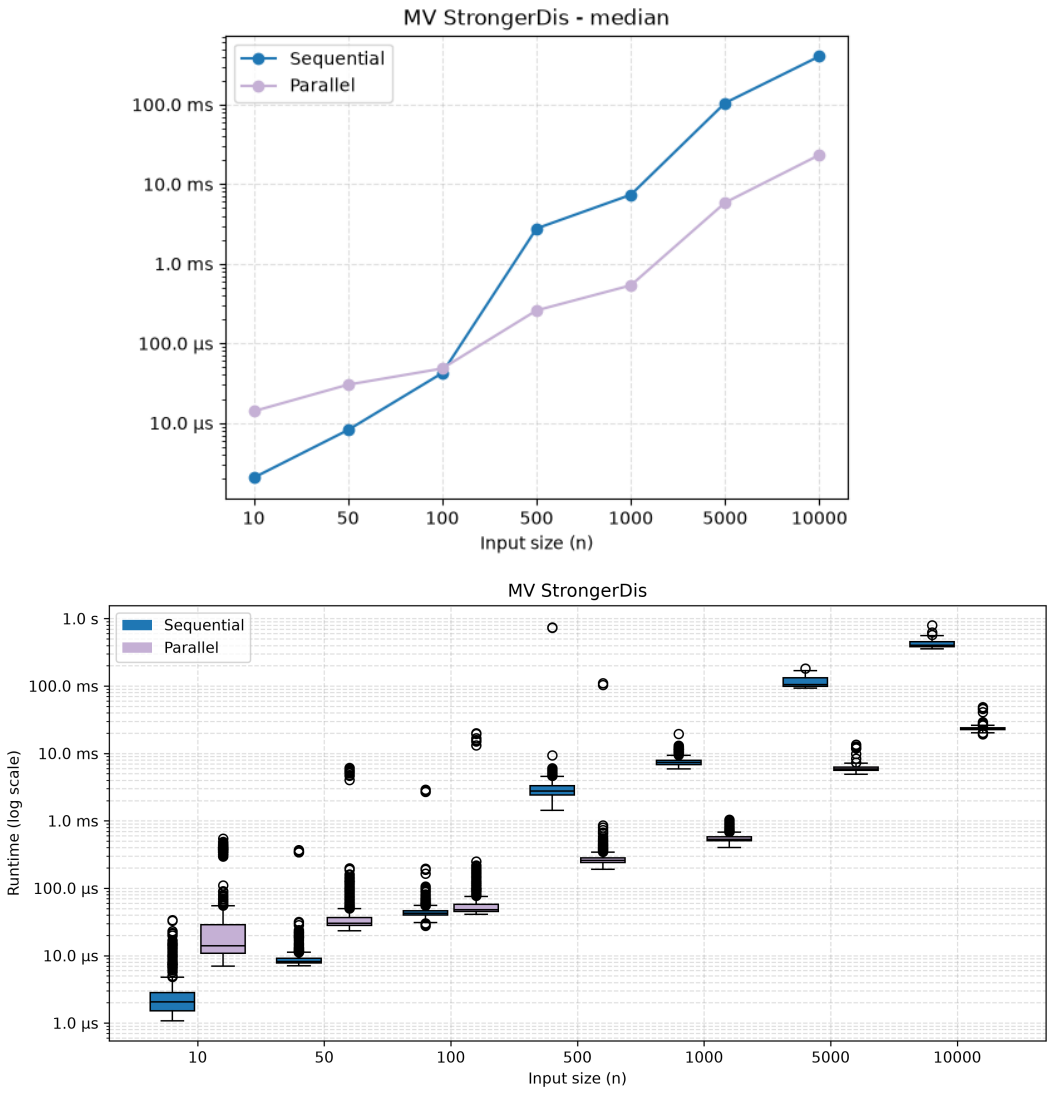

Figure 19: Comparison of the sequential and parallel implementations of the MV algorithm for solving StrongerDis.

C.2 EXP2.2: Space optimization

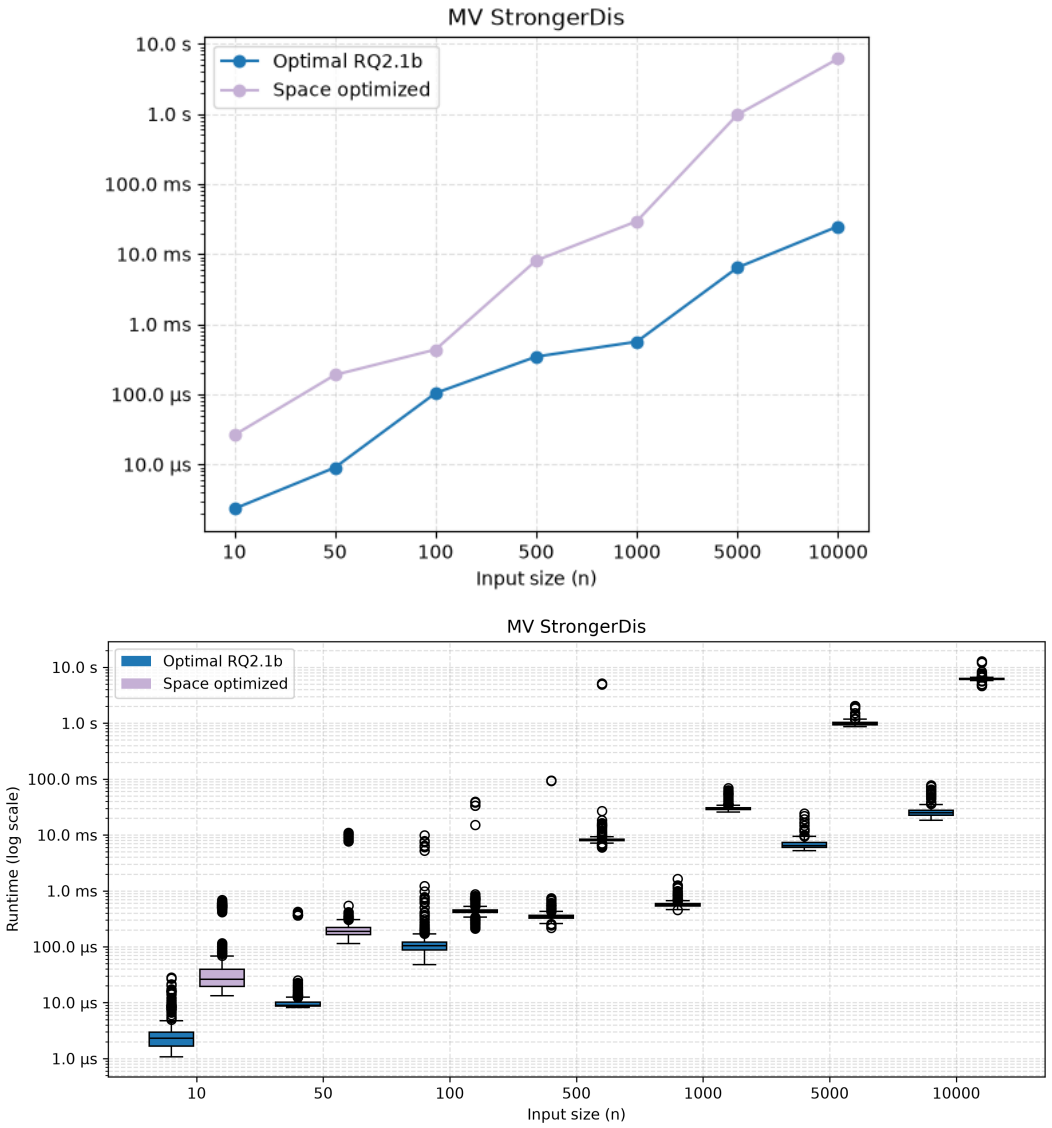

Figure 20: Comparison of the optimal version of EXP2.1 and the space-optimized version of the MV algorithm for solving StrongerDis.

C.3 EXP2.3: MM vs. MV

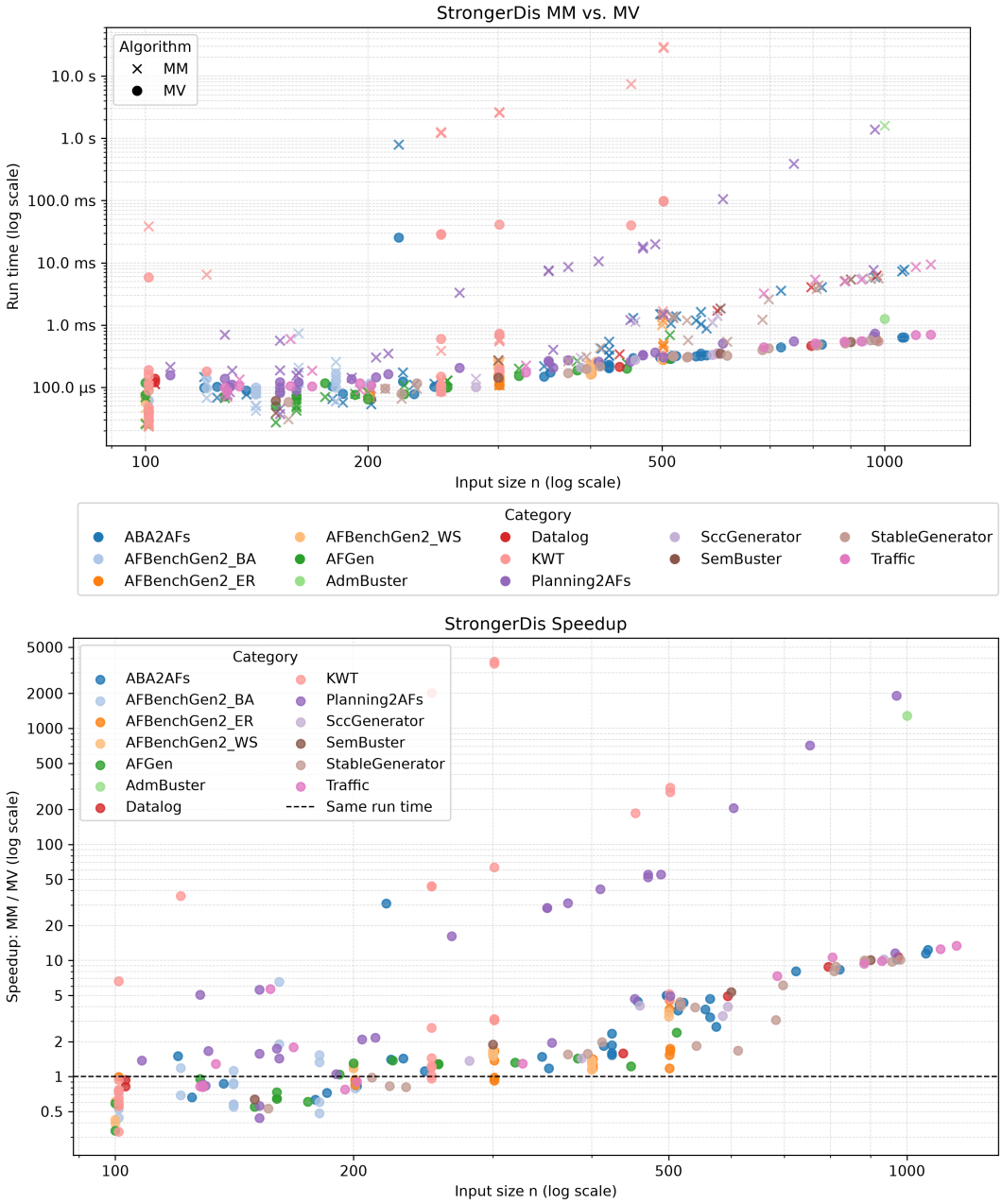

Figure 21: Comparison of the optimal MM and the MV implementations for solving StrongerDis.

D Result of Preliminary Test for MV StrongerDis Algorithm

The following graph shows the median run time of the MV algorithm for solving StrongerDis from a preliminary test. This preliminary test used all instances of the categories listed in the graph, and executed 1000 repetitions, selecting a random pair of arguments in each repetition. The hardware used was the same as for the main experiments.

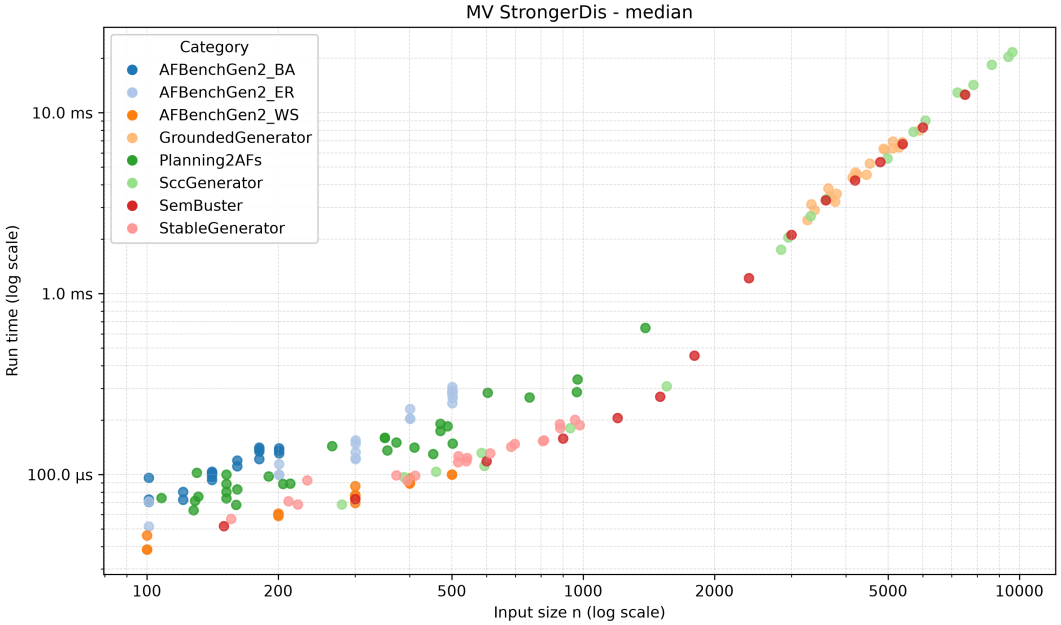

Figure 22: Median run time of the MV algorithm for solving StrongerDis in preliminary test.